



SmarTask - Sistema inteligente para geração automática de horários de trabalho

Universidade de Aveiro
Departamento de Eletrónica, Telecomunicações e Informática
Relatório Técnico
2025

Equipa

João Monteiro, Rafael Kauati
Gabriel Teixeira, Vitalie Bologna

Orientadores:

Amaro de Sousa,
Luís Seabra Lopes,
José Moreira

Índice

1 - Introdução.....	3
1.1 Contexto.....	3
1.2 Objetivos.....	3
1.3 Estrutura do documento.....	4
1.4 Caso de estudo.....	4
1.4.1 Motivação.....	4
1.4.2 Problema original.....	4
1.4.3 Restrições.....	5
1.4.4 Exigência de mínimos diários.....	6
2 - Estado de Arte.....	8
2.1 Tecnologias/serviços existentes.....	8
2.1.1 Análise generalizada.....	8
2.1.2 Resultado das análises.....	8
3 - Requisitos do sistema.....	10
3.1 Atores.....	10
3.2 Personas.....	10
3.3 Casos de uso.....	13
3.4 Requisitos funcionais.....	14
3.5 Requisitos não-funcionais.....	15
3.6 Arquitetura e tecnologias.....	16
3.6.1 Arquitetura.....	16
3.6.2 Tecnologias.....	17
3.7 Modelo de domínio.....	20
3.8 Pressupostos e Dependências.....	21
4 - Implementação.....	22
4.1 Frontend.....	22
4.1.1 Responsabilidade da Componente.....	22
4.1.2 Tecnologias e Ferramentas Utilizadas.....	22
4.1.3 Estrutura de Dependências.....	23
4.1.3.1 Dependências de Produção.....	23
4.1.3.2 Dependências de Desenvolvimento.....	23
4.1.4 Funcionalidades principais do frontend.....	24
4.1.4.1 Formulário de criação de horário.....	24
4.1.4.2 Validação de template de férias em função do número de dias de férias... 25	
4.1.4.3 Validação de Template de Férias com Base nos Empregados Existentes... 26	
4.1.5 Teste de vulnerabilidades.....	27
4.1.5.1 Teste de tentativa de acesso não autorizado.....	27
4.1.5.2 Teste de Cross-Site Scripting (XSS).....	28

4.1.6	Página principal e funcionalidades.....	29
4.1.6.1	Página de visualização de horários gerados.....	29
4.1.6.2	Home Page, processamento e estado dos horários.....	30
4.1.6.3	Página de comparação de soluções.....	31
4.1.6.4	Lista de Empregados e respectivas preferências.....	32
4.1.6.5	Página de listagem de horários.....	33
4.2	API.....	34
4.2.1	Estrutura da API.....	34
4.2.2	Gestão do estado das tarefas.....	36
4.2.3	Gestão de templates.....	37
4.2.4	Gestão de funcionários-equipas-preferências.....	37
4.2.6	Dependências.....	38
4.3	Módulos Python.....	38
4.3.1	Responsabilidade.....	38
4.3.2	RabbitMQClient.....	39
4.3.3	MongoDBClient.....	39
4.3.4	TaskManager.....	40
4.3.5	Analyser.....	40
4.4	Algoritmos.....	41
4.4.1	Responsabilidade.....	41
4.4.2	Dados de entrada.....	42
4.4.3	Restrições.....	42
4.4.4	Algoritmos com bons resultados.....	43
4.4.4.1	Programação Linear Inteira (ILP - Integer Linear Programming).....	43
4.4.4.2	Hill Climbing.....	45
4.4.4.3	Greedy Randomized.....	46
	Figura 28: Pseudo-Código Relativo ao Algoritmo.....	47
4.4.4.3	Greedy Randomized com Refinamento via Hill Climbing.....	47
4.4.5	Algoritmos com resultados insatisfatórios.....	49
4.4.5.1	Algoritmo Genético.....	49
4.4.5.2	Problema de Satisfação de Restrições (CSP – Constraint Satisfaction Problem). 50	
4.5	Message Broker.....	51
4.6	Base de Dados.....	51
4.7	Composição da aplicação.....	52
5	- Testes e resultados.....	52
5.1	- Configurações do testes.....	52
5.2	- Avaliação dos resultados.....	53
5.3	- Testes.....	53
5.3.1	- Testes de performance dos mínimos.....	53
5.3.2	- Testes de performance dos shift balance e two team preference level.....	54

5.3.3 - Testes de cumprimento de KPIs em função do tempo máximo.....	56
6 - Conclusão.....	57
6.1 - Conclusão e resumo do projeto.....	57
6.2 - Trabalho futuro.....	57
6.3 - Repositórios e Instruções.....	58

1 - Introdução

1.1 Contexto

O projeto insere-se no contexto de um problema recorrente e complexo enfrentado por empresas de média e grande dimensão: a afetação e gestão eficiente dos horários das suas equipas de trabalho. Este desafio envolve múltiplas variáveis, incluindo competências dos colaboradores, requisitos específicos dos postos de trabalho e o cumprimento da legislação laboral. Uma gestão eficaz desses horários pode melhorar a eficiência operacional, reduzir custos e promover o bem-estar dos recursos humanos.

Com o objetivo de automatizar este processo, o projeto propõe o desenvolvimento de métodos computacionais para a alocação anual de pessoal por turno e por equipas, com base numa formulação inicial simplificada do problema. As tarefas incluem a modelação lógica do problema, criação de uma plataforma de testes flexível, e implementação de algoritmos heurísticos, metaheurísticos e baseados em inteligência artificial para geração e otimização de horários.

Este trabalho está integrado no projeto SmarTask, que visa fornecer soluções de escalonamento automatizado para empresas dos setores do retalho e da saúde, como supermercados, cadeias de restaurantes e hospitais, onde a complexidade e variabilidade dos horários tornam essencial o uso de abordagens inteligentes para garantir eficiência e adaptabilidade.

1.2 Objetivos

Os objetivos centrais do projeto consistem, primeiramente, no desenvolvimento de métodos automáticos para a geração de horários de trabalho, com foco na alocação eficiente e justa de recursos humanos aos turnos e postos, respeitando as restrições operacionais e legais aplicáveis. Além disso, busca-se criar uma plataforma de testes que seja flexível e extensível, apta a executar processos de pesquisa e otimização, permitindo a análise comparativa entre diferentes métodos e possibilitando, no futuro, a integração

de novas abordagens. Por fim, o projeto visa implementar e avaliar algoritmos heurísticos, metaheurísticos e de inteligência artificial para a resolução do problema de escalonamento, sempre levando em consideração o cumprimento das normas laborais, de forma a garantir a geração de horários compatíveis com essas exigências.

1.3 Estrutura do documento

Este documento apresenta fundamentalmente cinco capítulos importantes, para além deste primeiro capítulo introdutório. No capítulo 2, é apresentado o estado da arte, detalhando o sistema existente. O capítulo 3 apresenta o processo de elicitação de requisitos, os requisitos funcionais e não funcionais do sistema e ainda a arquitetura e os seus diagramas. No capítulo 4, são descritos os detalhes de implementação tanto do frontend como backend, abordando ainda aspetos como segurança e backup. Depois, o capítulo 5 apresenta os testes de desempenho dos algoritmos desenvolvidos e usados no projeto final, realizados numa fase final do projeto, incluindo o método, configurações do teste e avaliação dos resultados obtidos. O último capítulo (6) oferece uma visão geral do trabalho desenvolvido, apresentando conclusões do mesmo e apontando direções para trabalho futuro. Ainda neste capítulo introdutório encontra-se destacado o caso de estudo base, fornecido pela empresa que solicitou o projeto, algo importante de ser considerado pois será de extrema importância para compreensão da implementação da aplicação final.

1.4 Caso de estudo

1.4.1 Motivação

A principal motivação deste projeto reside na necessidade de testar algoritmos de geração de horários de trabalho. O objetivo não é necessariamente gerar escalas finais para uso direto, mas sim testar vários algoritmos de geração e validar os seus resultados com base num conjunto de **KPIs** (Key Performance Indicators).

O propósito final é identificar o(s) algoritmo(s) mais eficaz(es) em termos de desempenho, escalabilidade e qualidade dos horários produzidos segundo os KPIs definidos. Pretende-se também compreender os limites qualitativos e de escalabilidade de cada algoritmo, mediante a variação de restrições e do volume de dados de entrada.

1.4.2 Problema original

Para iniciar o planejamento do projeto, a empresa solicitante do projeto forneceu um documento contendo um caso de uso representativo do problema a resolver: um horário contendo informações sobre o turno (manhã ou tarde) que um dado funcionário irá cumprir no dia x (inicialmente somente para 1 mês) ou representação de dias de folga ou férias. No caso do funcionário trabalhar num dado dia, deve ser indicado, de forma explícita, a equipa para a qual ele vai trabalhar nesse mesmo dia, uma vez que um funcionário pode pertencer a uma ou mais equipa ao mesmo tempo.

No que diz respeito às equipas, cada funcionário que possa trabalhar em mais do que uma deve ter as suas preferências de alocação especificadas, ou seja, a ordem pela qual prefere trabalhar em cada equipa.

exemplo

☆

📁

🔍

Alterações salvas no Drive

Arquivo

Editar

Ver

Inserir

Formatar

Dados

Ferramentas

Extensões

Ajuda

🔍

Menus

↶

↷

🖨

100%

R\$

%

0.00

123

Arial

11

B

I

🔗

A

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

🔗

Figura 1: Caso de Estudo

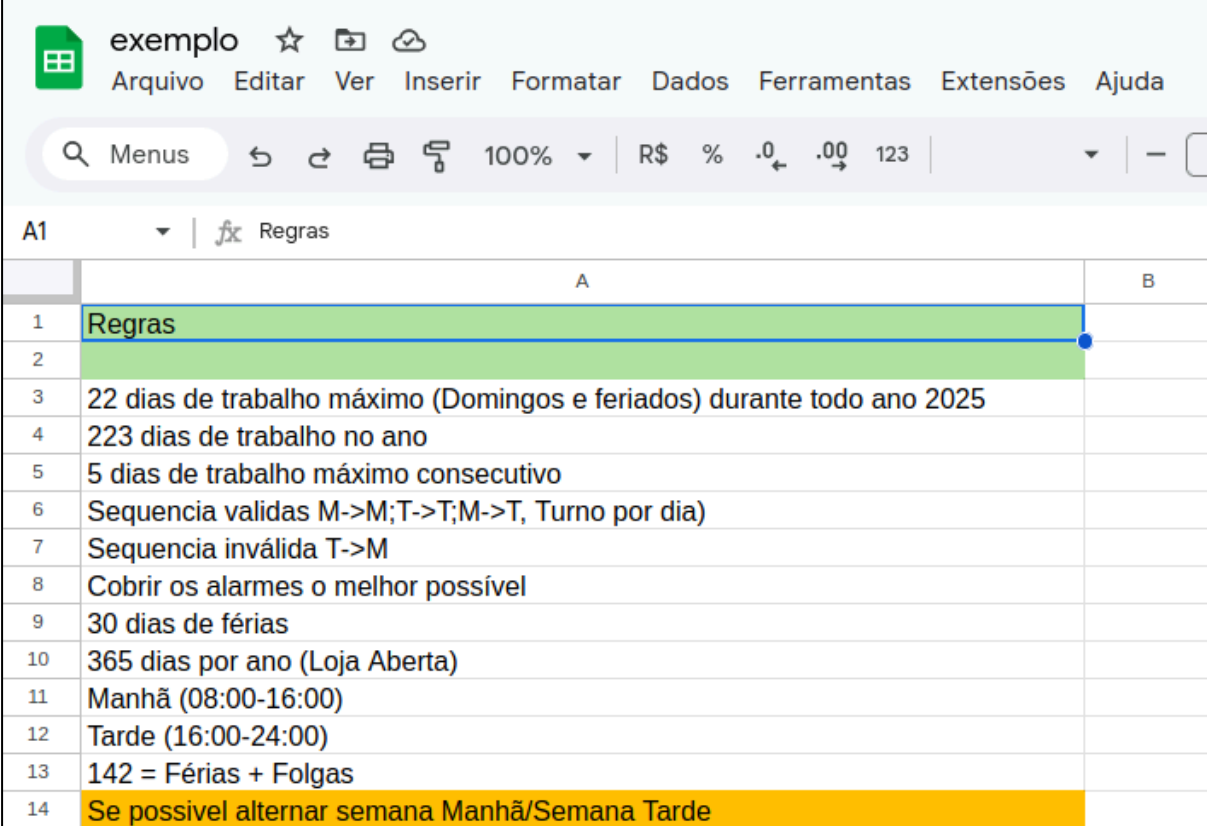
Este cenário, embora simplificado (apenas um mês), foi utilizado como base para o planejamento dos algoritmos a desenvolver, bem como para a elicitação dos requisitos funcionais do sistema final. Este sistema irá suportar toda a configuração, execução e monitorização dos testes necessários.

1.4.3 Restrições

Para assegurar que os horários gerados estão de acordo com as normas e legislações laborais e com as políticas internas da empresa, foi definido um conjunto de **restrições obrigatórias**, que devem ser cumpridas na geração de qualquer horário, independentemente dos tipos de dados de entrada ou quantidade dos mesmos. As principais restrições são as que se seguem:

- Exatamente **223 dias de trabalho por ano** por funcionário;
- No **máximo 22 domingos ou feriados** trabalhados;
- No **máximo 5 dias consecutivos** de trabalho;

- **Proibição de transições T→M** em dias consecutivos (trabalhar à tarde e no dia seguinte de manhã);
- Considera-se que cada funcionário **trabalha apenas um turno por dia** (manhã ou tarde);
- Ter **142 dias sem trabalho por ano**, dos quais **30 são de férias**;
- **Obrigatoriedade de folgas nos dias marcados como férias.**



	A	B
1	Regras	
2		
3	22 dias de trabalho máximo (Domingos e feriados) durante todo ano 2025	
4	223 dias de trabalho no ano	
5	5 dias de trabalho máximo consecutivo	
6	Sequencia validas M->M;T->T;M->T, Turno por dia)	
7	Sequencia inválida T->M	
8	Cobrir os alarmes o melhor possível	
9	30 dias de férias	
10	365 dias por ano (Loja Aberta)	
11	Manhã (08:00-16:00)	
12	Tarde (16:00-24:00)	
13	142 = Férias + Folgas	
14	Se possivel alternar semana Manhã/Semana Tarde	

Figura 2: Restrições estipuladas pela empresa

Estas restrições são cruciais para a avaliação dos algoritmos, sendo utilizadas como **indicadores de qualidade** no conjunto de KPIs (Key Performance Indicators) definidos para análise.

1.4.4 Exigência de mínimos diários

Além do cumprimento das restrições, é necessário assegurar que determinados **mínimos operacionais diários** são cumpridos, por turno e por equipa.

Estes mínimos são definidos para cada um dos **365 dias do ano**, por equipa (neste cenário: A e B) e por turno (manhã e tarde). Cada falha no cumprimento de um mínimo (por exemplo, se num dia estiverem alocados menos colaboradores do que o mínimo definido para o turno da manhã da equipa A) será contabilizada como uma falha de KPI.

O valor ideal deste KPI é **zero**, ou seja, nenhuma falha de mínimos ao longo do ano. No entanto, este objetivo deve ser atingido **sem comprometer** os restantes KPIs relacionados com restrições laborais, sendo, por isso, preferível cumprir restrições legais em detrimento dos mínimos impostos.

				22	23	24	25	26	27	28	29	30	31	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	
				D	S	T	Q	Q	S	S	D	S	T	Q	Q	S	S	D	S	T	Q	Q	S	S	D	S	T	Q	Q	S	S	D	
Equipa A	Minimo	M					1	2	2	2	2	2	1	2	2	2	2	2	2	1	2	2	2	2	1	2	2	2	2	2	1	2	1
	Ideal	M					2	2	2	2	2	3	2	2	2	2	2	3	2	2	2	2	2	3	2	2	2	2	2	2	3	2	2
	Minimo	T					1	2	2	2	2	2	1	2	2	2	2	2	2	1	2	2	2	2	2	1	2	2	2	2	2	1	2
	Ideal	T					2	2	2	2	2	3	3	2	2	2	2	3	3	2	2	2	2	3	3	2	2	2	2	2	3	2	2
Equipa B	Minimo	M					1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
	Ideal	M					1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
	Minimo	T					1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
	Ideal	T					1	1	1	1	1	2	2	1	1	1	1	2	2	1	1	1	1	1	1	2	2	1	1	1	1	2	2

Figura 3: mínimos e ideais diários dados

2 - Estado de Arte

2.1 Tecnologias/serviços existentes

2.1.1 Análise generalizada

O estado da arte na área de ferramentas de geração automática de horários de trabalho inclui soluções comerciais como **Deputy**, **When I Work** e **7shifts**. Estas ferramentas demonstram avanços relevantes, no entanto também apresentam limitações que justificam o desenvolvimento de abordagens mais flexíveis e adaptáveis.

O Deputy destaca-se pela automação do cumprimento de normas laborais e pela otimização de custos com mão de obra, oferecendo geração personalizada de horários. No entanto, a eficácia da inteligência artificial (IA) utilizada requer afinação contínua, e os horários iniciais podem apresentar baixa precisão. Além disso, a complexidade da configuração inicial e a curva de aprendizagem para os gestores dificultam a sua adoção em contextos organizacionais mais amplos.

O When I Work aposta numa interface simples com modelos de horário fáceis de utilizar, sendo adequada para pequenas e médias equipas. Apesar disso, também enfrenta limitações na escalabilidade para planeamentos mais complexos e de longo prazo, e, tal como as demais, não permite a comparação entre diferentes métodos de geração de horários, limitando a capacidade de análise da eficácia das soluções obtidas.

O 7shifts, voltada especificamente para o setor da restauração, oferece forte conformidade com leis laborais e mecanismos eficazes de controle de custos. Utiliza IA para prever necessidades de pessoal com base em padrões de vendas e procura, mas é pouco flexível fora do seu nicho e, à semelhança das outras ferramentas, não permite a avaliação comparativa de diferentes abordagens algorítmicas.

2.1.2 Resultado das análises

As tecnologias analisadas: **Deputy**, **When I Work** e **7shifts**, oferecem soluções relevantes para a gestão de horários, mas apresentam limitações significativas quando comparadas com os objetivos delineados neste projeto. Nenhuma delas permite a comparação entre diferentes métodos de geração de horários, o que impede a avaliação da eficácia relativa de algoritmos distintos, uma funcionalidade central na plataforma a ser desenvolvida.

Além disso, essas soluções não oferecem suporte a métodos personalizados de pesquisa e otimização, como algoritmos heurísticos, metaheurísticos ou baseados em inteligência artificial ajustáveis ao contexto específico de cada organização. A

flexibilidade para incorporar novas abordagens ou restrições específicas, como a resposta rápida a indisponibilidades ou a configuração de regras complexas de trabalho é limitada ou inexistente.

Por fim, as ferramentas existentes tendem a ter alcance setorial restrito (como no caso do 7shifts para restauração) ou dificuldades de escalabilidade e adaptação a contextos mais complexos e dinâmicos. Em contraste, o projeto proposto visa criar uma plataforma extensível e generalista, capaz de ser aplicada em diversos setores com necessidades distintas.



Figura 4 : deputy



Figura 5 : 7shifts



Figura 6 : when i work

3 - Requisitos do sistema

Esta seção apresenta os requisitos gerais do sistema final, bem como especificações da arquitetura, tecnologias e modelos de dados usados na implementação do projeto.

Todos os tópicos aqui desenvolvidos resultaram de discussões com os docentes orientadores do projeto, com base numa visão inicial dos objetivos e no escopo previamente definido.

3.1 Atores

Dado que este sistema fornece um serviço altamente específico, é expectável que seja disponibilizado por empresas especializadas na gestão de horários a outras organizações (clientes finais). Assim sendo, decidimos dividir o sistema em dois diferentes atores:

- **Administrador:** este ator é o maior responsável do sistema como um todo. Deverá ser um trabalhador da empresa que fornece o sistema. O mesmo é responsável por adicionar novos métodos (algoritmos) desenvolvidos ao sistema e pelo acompanhamento da sua performance.

- **Gestor de Equipa:** este ator faz parte da empresa cliente que adota o SmarTask para geração de horários. É o responsável pela atribuição de horários aos restantes trabalhadores. Para tal faz uso do sistema, de modo a economizar tempo e facilitar as suas funções. Deve ser responsável por tirar conclusões dos horários (com os diversos auxílios do sistema como, por exemplo, os KPIs) para, posteriormente, atribuir a melhor alternativa possível.

3.2 Personas

De igual modo, definimos duas personas diferentes, para motivar a divisão do sistema em dois atores.

- Luís, um **administrador** com 37 anos de idade.
- Catarina, uma **gestora de equipa** com 45 anos de idade.

Luís



Figura 7: Administrador - Luís

O Luís, funcionário da empresa distribuidora do serviço, é responsável por administrar o sistema oferecido.

Dado que a sua empresa está numa fase de expansão de serviços, um novo sistema foi desenvolvido para gerar horários para um número de empregados de forma automática. Assim sendo, Luís é o encarregado da gestão deste sistema. Especificamente, ele é responsável por adicionar ou remover diferentes algoritmos para gerar horários.

Deste modo, ele necessita de uma ferramenta que lhe permita gerir os algoritmos disponíveis no sistema, assim como monitorizar a performance de cada um. É assim essencial que o mesmo tenha acesso aos resultados dos horários gerados para comparar diferentes resultados, de modo a perceber quais são mais capazes.

Catarina



Figura 8: Gestora - Catarina

Catarina é uma funcionária e gestora de equipa de 45 anos. Por isso mesmo, é responsável por criar e atribuir horários aos seus colegas.

Com o desenvolvimento da sua empresa, tem existido um aumento substancial no número de funcionários nos quadros da mesma. Por isso mesmo, tem-se tornado impossível atribuir estes horários manualmente.

Ela necessita de uma ferramenta que a ajude a garantir horários para todos os seus pares, da forma mais rápida e justa possível. Dentro desse mesmo sistema necessita de uma plataforma que lhe permite editar equipas, adicionar mapas de férias e mínimos por equipa, assim como visualizar o progresso da geração de diferentes soluções, de forma a manter controlo sobre este processo.

3.3 Casos de uso

De seguida, uma apresentação dos casos de uso levantados para o nosso sistema:

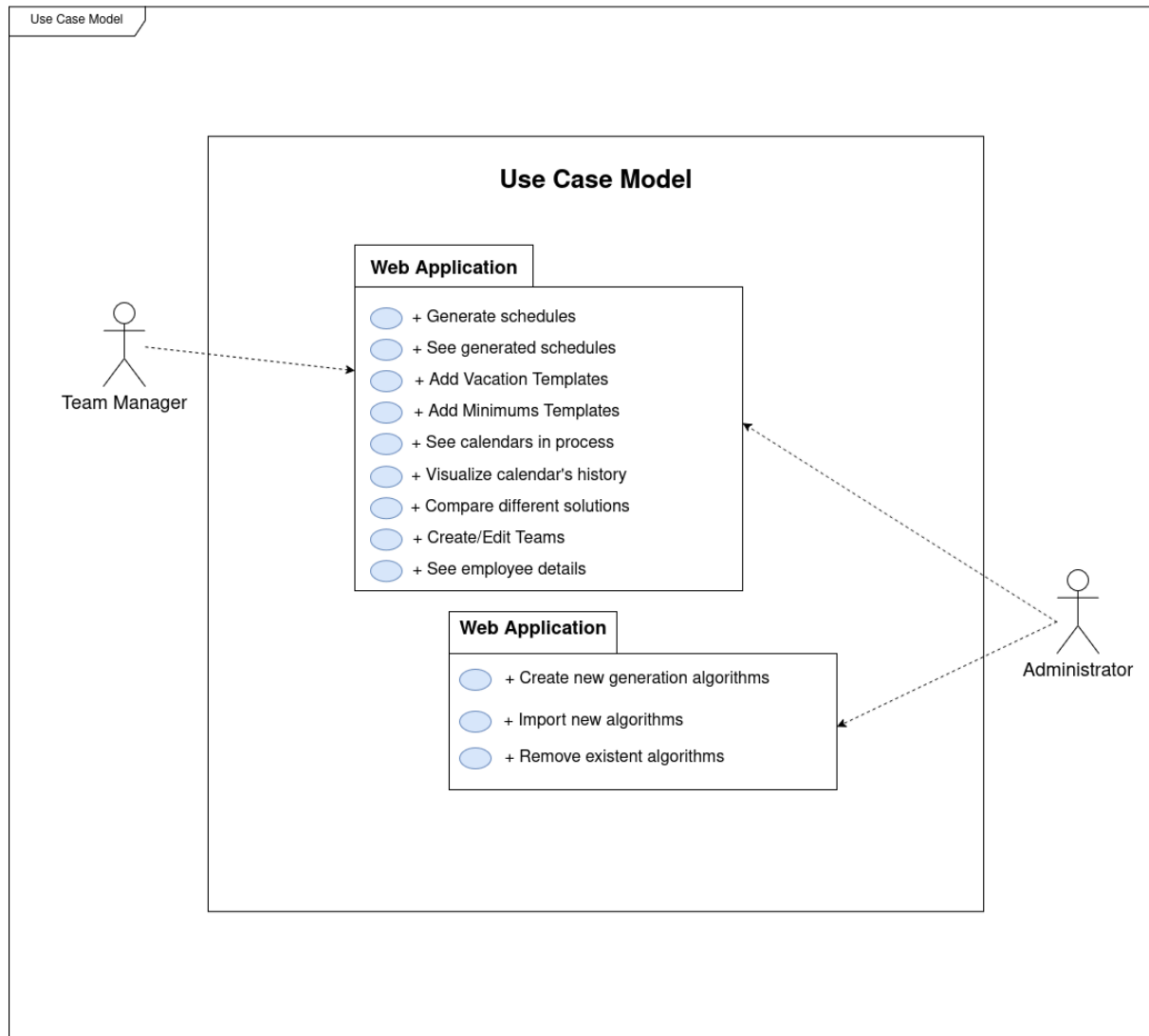


Figura 9: Use Case Model

Começando pelo **gestor de equipa**, o maior foco do mesmo, como mencionado anteriormente, é a geração e visualização de horários para um dado conjunto de funcionários. Assim sendo, é-lhe possível:

- Dado um conjunto de funcionários definidos (podendo estar separados por equipas), um mapa de férias para cada um e ainda um template com o número mínimo e ideal de funcionários necessários para cada dia, turno e equipa, gerar horários fazendo uso de um dos métodos existentes no sistema;
- Visualizar todos os horários previamente gerados, assim como toda a informação usada para a geração do mesmo (equipas, férias e mínimos). Para além disso, é também possível ver uma análise geral dos resultados de cada uma geração, verificando se houve alguma métrica que tenha falhado;

- Adicionar os mapas de férias e de mínimos referidos anteriormente, de forma a definir necessidades empresariais em dados dias, respeitando o direito a um período de descanso de cada trabalhador;
- Visualizar o estado atual de processamento de uma nova solução, isto é, no sistema é possível visualizar se um horário está a ser gerado ou se, por sua vez, já terminou essa geração, indicando se foi sucedido ou se, por alguma razão, falhou;
- Adicionar novos funcionários ao sistema que podem ser atribuídos a uma ou mais equipas (sendo necessário definir as preferências para cada um), sendo também possível criar novas equipas em caso de necessidade;
- Comparar diferentes soluções geradas: dado dois diferentes horários, gerados por qualquer algoritmo disponível no sistema, é possível comparar os resultados obtidos, visualizando essa comparação, baseada em KPIs (Key Performance Indicators), auxiliando o utilizador a definir a melhor solução.

Já o **administrador**, dadas as necessidades mencionadas anteriormente, necessitará acesso a todas estas funcionalidades “básicas” do sistema, maioritariamente para testagem do mesmo. Será também capaz de:

- Criar novos algoritmos que sejam capazes, dadas as especificações envolvidas, de gerar uma solução minimamente válida para o sistema;
- Importar esses novos métodos de forma prática, de modo a poder integrá-los no sistema, assim como, remover possíveis métodos que se revelem incapazes de satisfazer as necessidades envolvidas.

3.4 Requisitos funcionais

Abaixo encontra-se a lista de requisitos de funcionais necessários para que a aplicação final alcance os seus objetivos:

1. **Geração de horários**

O sistema deve permitir ao utilizador solicitar a geração de um novo horário com base nas restrições e parâmetros definidos.

2. **Otimização dos horários gerados**

O sistema deve incorporar mecanismos de otimização que melhorem as soluções inicialmente geradas, com base em critérios como equilíbrio da carga de trabalho, cobertura mínima por turno e cumprimento da legislação laboral.

3. **Consulta e recuperação de horários anteriores**

O sistema deve disponibilizar funcionalidades para consultar, listar e recuperar todos os horários previamente gerados e armazenados na base de dados, possibilitando análises históricas e comparativas.

4. **Integração de um conjunto adaptável de algoritmos de geração**

O sistema deve suportar a implementação modular de diferentes algoritmos de geração de horários (heurísticos, metaheurísticos e baseados em IA), permitindo a sua adaptação e evolução conforme o contexto da organização.

5. **Atualização em tempo real do processo de geração de horários**

A plataforma deve oferecer mecanismos de monitorização em tempo real, fornecendo feedback contínuo sobre o estado e progresso da geração e otimização dos horários.

6. **Comparação de múltiplos horários com base em indicadores de desempenho (KPIs)**

O sistema deve possibilitar a comparação entre diferentes soluções geradas, com base em indicadores de desempenho quantitativos (por exemplo, número de violações de restrições, equilíbrio de turnos, satisfação de preferências), permitindo a seleção da solução mais eficiente.

7. **Gestão dinâmica da introdução de novos métodos de geração**

O sistema deve permitir a integração e ativação dinâmica de novos métodos de geração e otimização de horários, sem necessidade de reconfiguração estrutural da plataforma, assegurando extensibilidade e escalabilidade do ambiente de testes.

3.5 Requisitos não-funcionais

Abaixo encontra-se a lista de requisitos de não-funcionais necessários para que a aplicação final alcance os objetivos propostos:

1. **Desempenho e escalabilidade no processo de geração de horários**

O sistema deve apresentar elevada eficiência computacional na geração e otimização de horários, garantindo tempos de resposta compatíveis com o uso prático em ambientes corporativos. Deve ainda ser escalável, suportando um aumento, por exemplo, no número de colaboradores.

2. **Monitorização do sistema e visualização de métricas operacionais**

A arquitetura da plataforma deve incluir mecanismos de monitorização contínua do seu estado e desempenho, bem como ferramentas de recolha, agregação e visualização de métricas operacionais (Key Performance Indicators), com o objetivo de facilitar a análise, a manutenção e a melhoria contínua dos algoritmos.

3. Alta disponibilidade e fiabilidade durante o processo de geração de horários

O sistema deve assegurar elevados níveis de disponibilidade e fiabilidade ao longo de todo o processo de geração e otimização de horários, minimizando falhas, interrupções e perda de dados, não só para garantir uma boa usabilidade do mesmo, mas também para garantir que processos não são interrompidos durante a sua execução.

3.6 Arquitetura e tecnologias

De seguida, é apresentada a arquitetura geral do sistema, representada no diagrama. Serão assim, apontados os 5 módulos fundamentais que constituem a estrutura do sistema.

3.6.1 Arquitetura

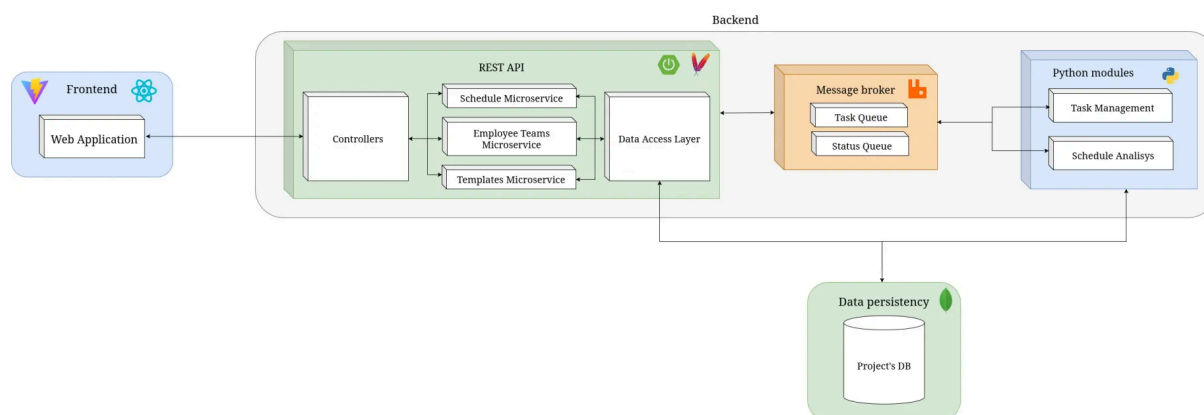


Figura 10: Arquitetura

- **Web application (Frontend):** O serviço Web Application fornece uma interface gráfica ao utilizador final, de forma a facilitar a utilização da plataforma. Este serviço é composto por um conjunto de páginas web. Estas páginas estão encarregues de comunicar com os restantes serviços por via de pedidos HTTP, como por exemplo, comunicação com os serviços da API desenvolvida, quando tal é pedido pelo utilizador. Após receber as respostas destes serviços, os dados de retorno são então processados pelas páginas de forma a distribuir graficamente o estado atual da plataforma e das suas alterações através das páginas web.

- **Application Programming Interface:** Este componente permite que os pedidos requisitados pelo utilizador na interface gráfica sejam cumpridos. Aqui, os controllers comunicam com o frontend, que através de um conjunto de endpoints, reencaminham os pedidos para os serviços definidos na API, onde toda a lógica para a resolução desses mesmos pedidos é feita.
- **Algorithm's Functional Integration modules:** Conjunto de módulos desenvolvidos em Python que fornecem suporte à integração funcional dos algoritmos de geração de horários com o restante sistema, desde o uso para execução destes algoritmos até à análise de resultados obtidos pelos mesmos. O módulo de gestão de tarefas (Task Management), recebe pedidos com os dados necessários em JSON por meio do Message Broker usado e lança um processo de geração de horário como um worker, que executa o algoritmo escolhido na sua main thread. O módulo de análise (Schedule Analysis) também recebe pedidos por meio do Message Broker para realização da análise de horários. Ambos os módulos armazenam os resultados dos seus respectivos processos na base de dados do sistema.
- **Message Broker:** Este módulo foi desenvolvido em RabbitMQ. Para assegurar comunicação assíncrona entre a realização de tarefas (bem como a atualização do estado de cada tarefa) da API e a execução das mesmas tarefas nos seus módulos python, é utilizado um Message Broker com queues responsáveis para cada tipo de mensagem, assegurando a comunicação assíncrona entre estes dois componentes da arquitetura, permitindo independência operacional entre os mesmos, execução desacoplada e controlo independente dos estados entre cada componente, evitando qualquer tipo de bloqueio (non-blocking) no fluxo das operações de ambos os módulos
- **Persistência de dados:** Visto que existe necessidade de assegurar a persistência de dados, principalmente no que diz respeito aos horários gerados, seus resultados obtidos e ainda os seus metadados, foi utilizado como destino de persistência de dados principal uma base de dados MongoDB.

3.6.2 Tecnologias

Abaixo, segue-se destacado a lista de tecnologias (linguagens e frameworks) usadas em cada componente da arquitetura, bem como os principais motivos pela escolha de cada uma dessas tecnologias :

- **Spring Boot (API) :** A escolha do Spring Boot como framework para o backend REST API do projeto justifica-se pela sua forte adequação ao desenvolvimento de microsserviços, oferecendo uma estrutura modular,

escalável e de fácil manutenção. Ele facilita a criação de APIs REST organizadas e robustas, ideais para gerir de forma separada diferentes aspectos do sistema como configuração de dados, geração de horários, templates e controlo de tarefas. Com integração nativa com ferramentas como RabbitMQ, o Spring Boot permite comunicação assíncrona eficiente com os módulos de execução dos algoritmos, suportando de forma fluida o ciclo de vida das tarefas.

- **Python e bibliotecas** (módulos python) : A escolha do Python para os módulos responsáveis pela execução e suporte funcional dos algoritmos de geração de horários é justificada pela sua flexibilidade, simplicidade e ampla oferta de bibliotecas voltadas à inteligência artificial, otimização e manipulação de dados. Nesse projeto, o Python atua como núcleo funcional dos algoritmos, isolado do restante do sistema, favorecendo uma arquitetura orientada a microsserviços. Com o uso de bibliotecas como **Pika** para comunicação assíncrona via RabbitMQ e PyMongo para acesso ao banco de dados MongoDB, os módulos **Python** conseguem processar pedidos de geração de horários de forma desacoplada, eficiente e escalável. Essa separação permite que algoritmos distintos sejam facilmente integrados ou substituídos, sem impactar o restante da aplicação, oferecendo grande flexibilidade para testes, extensões e evolução do sistema ao longo do tempo.
- **React** (Frontend) : A escolha de React para o desenvolvimento da interface gráfica do projeto SmarTask justifica-se pela sua capacidade de construir aplicações web modernas, escaláveis e com excelente experiência de utilizador. Com uma arquitetura baseada em componentes reutilizáveis, o React facilita a organização do código e a manutenção da interface, além de permitir fácil integração com a API REST do backend. Em uma arquitetura de microsserviços, o React funciona como um cliente desacoplado, capaz de consumir dados de diferentes serviços de forma eficiente e responsiva. O uso de tecnologias complementares como React Router DOM para navegação SPA, Tailwind CSS para estilizar rápida e consistentemente e Vite para desempenho no desenvolvimento torna o frontend altamente produtivo, flexível e preparado para evoluir junto com o restante sistema.
- **RabbitMQ** (Message Broker) : A escolha do RabbitMQ como Message Broker para este projeto foi motivado pela sua simplicidade, confiabilidade e aderência ao protocolo AMQP, que oferece uma comunicação assíncrona robusta entre a API e os módulos Python. Em comparação com outras opções como Apache Kafka, o RabbitMQ mostrou-se mais adequado por ser mais leve, fácil de configurar e melhor

adaptado a casos de uso com **baixa a média taxa de mensagens e forte necessidade de confirmação de entrega**, como é o caso dos pedidos de geração de horários e atualizações de status de execução. Além disso, o RabbitMQ oferece suporte nativo a filas com roteamento flexível, entrega garantida e controlo fino sobre mensagens, o que facilita a coordenação entre serviços em um ambiente com múltiplos containers. Sua integração simples com bibliotecas tanto em Java quanto em Python permite uma comunicação fluida entre os microserviços, tornando-o a escolha ideal para esse cenário de orquestração entre backend e módulos de execução.

- **MongoDB** (Base de dados) : A escolha do MongoDB como base de dados para o projeto justifica-se pela sua flexibilidade no armazenamento de dados semiestruturados e pela facilidade de modelar entidades complexas como calendários, preferências de funcionários e metadados associados a horários gerados. Como banco NoSQL orientado a documentos, o MongoDB permite armazenar estruturas como listas de listas de strings (usado para representação de calendários) de forma natural e eficiente, sem a rigidez de esquemas fixos exigidos por bancos relacionais. Isso facilita a evolução do modelo de dados conforme o projeto cresce e permite incluir novos campos ou alterar formatos sem grandes mudanças estruturais. Além disso, a sua capacidade de organizar os dados em coleções distintas e consultar documentos de maneira rápida o torna ideal para operações frequentes de leitura e escrita feitas tanto pela API quanto pelos módulos Python. A integração fluida com bibliotecas como PyMongo reforça sua utilidade como solução leve, escalável e altamente compatível com a arquitetura adotada.

3.7 Modelo de domínio

De seguida, apresentamos o modelo de domínio final que permite ter uma visão globalizada do funcionamento do sistema e das necessidades do mesmo.

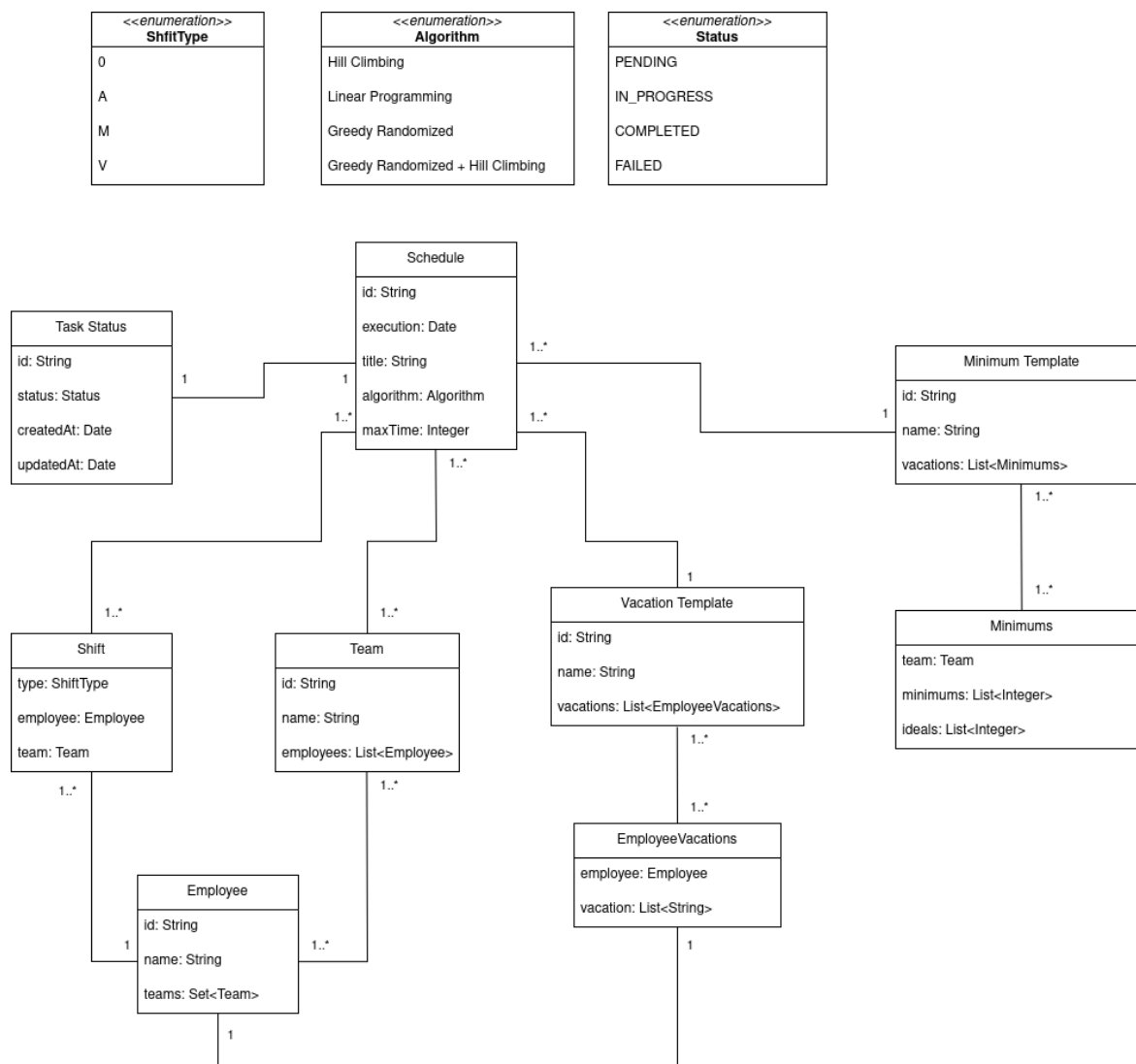


Figura 11: modelo de domínio

O foco central não podia deixar de ser o horário, já que esse foi o objetivo primário no desenvolvimento deste sistema. É guardado o momento em que cada um foi requisitado (campo execution), uma vez que é possível definir um tempo máximo de execução, tempo esse que se for excedido obriga à paragem da geração do dado horário.

A cada horário está associado um status, com 4 tipos diferentes: PENDING, IN_PROGRESS, COMPLETED e FAILED. Isto permite ao utilizador entender o estado atual da execução de um processo, sendo especialmente necessário para identificar soluções em construção ou que tenham falhado.

Para cada horário definido é necessário existir uma ou mais equipas, constituídas por um conjunto de funcionários, que podem, por sua vez, trabalhar em mais do que uma equipa (definindo assim preferências para esses casos). Cada horário é constituído

por um conjunto de turnos que está associado a um trabalhador, mantendo referência à equipa a que cada turno se refere. Cada um pode ser constituído por um de 4 valores diferentes: 0, A, M e V referentes a dias de folga, turno da tarde, turno da manhã ou dia de férias, respectivamente.

Adicionalmente, é ainda necessário que sejam passados um mapa de requisitos (onde é especificado, para cada dia do ano, o número de trabalhadores mínimo e ideal para cada turno e cada equipa) e também um mapa de férias, com informações dos dias de férias associados a cada um dos funcionários elegível para o horário requisitado.

3.8 Pressupostos e Dependências

Tendo em conta que a aplicação foi desenvolvida com o propósito de realizar testes e validações em ambiente de desenvolvimento, foram definidos os seguintes pressupostos e dependências:

- A execução da aplicação requer apenas a instalação local das tecnologias necessárias, nomeadamente as linguagens, bibliotecas e frameworks envolvidas, as quais podem ser facilmente geridas através de ambientes virtuais.
- A orquestração dos serviços que compõem o sistema (frontend, backend, módulos de execução, base de dados, etc.) é realizada com recurso ao **Docker Compose**, sendo este o único requisito fundamental para a execução local do sistema. A instalação do docker-compose (e, por consequência, do Docker) é suficiente para lançar todos os componentes em containers.
- O sistema não depende de serviços externos para funcionalidades críticas, como APIs de terceiros. Assim, após a primeira construção da aplicação (build inicial), que necessita de ligação à internet para descarregar imagens e dependências, o sistema poderá ser utilizado **offline**, sem qualquer ligação adicional à rede.
- A interface gráfica (frontend) é automaticamente exposta através de um servidor web local, acessível via browser, facilitando o processo de validação e interação com a aplicação por parte dos utilizadores.

4 - Implementação

4.1 Frontend

4.1.1 Responsabilidade da Componente

O módulo Frontend é responsável por fornecer a interface gráfica da aplicação Smartask, permitindo a interação dos utilizadores com as funcionalidades do sistema. Este componente atua como ponte entre o utilizador final e o restante sistema, apresentando visualmente os dados obtidos da API e organizando as ações disponíveis de forma acessível e intuitiva. A componente implementa a lógica de navegação entre páginas, gestão de estado local e exibição dos dados recebidos do backend.

4.1.2 Tecnologias e Ferramentas Utilizadas

A construção do frontend foi orientada por princípios de escalabilidade, manutenção e desempenho. Para tal, foram selecionadas tecnologias modernas e amplamente adotadas no ecossistema de desenvolvimento web:

- O React.js foi a biblioteca JavaScript central na construção da interface da aplicação. Através de uma arquitetura baseada em componentes reutilizáveis, o React permitiu modularidade, reutilização de código e melhor organização estrutural do projeto.
- O React Router DOM foi utilizado para implementar o roteamento da aplicação do tipo SPA (Single Page Application), permitindo transições rápidas entre páginas sem carregamentos completos, o que contribui para uma experiência de utilizador fluida.
- O Tailwind CSS foi adotado como framework de estilização, permitindo a aplicação eficiente e consistente de estilos, além de facilitar o design responsivo e a prototipação rápida.
- O Vite foi a ferramenta escolhida para o empacotamento e desenvolvimento da aplicação. Sua arquitetura moderna proporcionou tempos de inicialização reduzidos e uma experiência de desenvolvimento mais ágil e responsiva.
- O MUI (Material-UI) foi a biblioteca de componentes visuais utilizada com base nas diretrizes do Material Design, oferecendo uma interface moderna, coesa e acessível, com componentes prontos para uso e personalizáveis.

4.1.3 Estrutura de Dependências

A aplicação frontend baseia-se num conjunto de dependências organizadas em dois grupos principais:

- **Dependências de Produção**, utilizadas diretamente na aplicação em tempo de execução;
- **Dependências de Desenvolvimento**, usadas durante o processo de construção, testes e análise de código.

4.1.3.1 Dependências de Produção

No grupo das dependências de produção, destacam-se as bibliotecas e frameworks essenciais para a funcionalidade e a experiência do utilizador final. React e React DOM constituem o núcleo da construção da interface, sendo responsáveis pela renderização eficiente dos componentes e pela criação de uma aplicação interativa e reativa. Para viabilizar a navegação em formato SPA (Single Page Application), foi utilizado o React Router DOM, que possibilita a transição entre diferentes rotas sem a necessidade de recarregar a página, assegurando uma experiência fluida.

A comunicação assíncrona com o backend é realizada por meio do **Axios**, um cliente HTTP leve e robusto que permite o envio e receção estruturada de dados entre o frontend e a API. No que se refere à estilização da interface, a aplicação faz uso do **Tailwind CSS**, um framework de utilitários que facilita a criação de layouts responsivos e visualmente consistentes. Além disso, a biblioteca **MUI (Material-UI)** foi integrada ao projeto para oferecer um conjunto de componentes visuais pré-construídos, baseados nas diretrizes do Material Design, promovendo uma interface moderna e acessível.

Já a comunicação em tempo real é suportada pelas bibliotecas **STOMP.js** e **SockJS**, responsáveis pela implementação de canais WebSocket, permitindo atualizações dinâmicas da interface em resposta a eventos do servidor.

O suporte visual e interativo é reforçado pelas bibliotecas **Lucide React**, **React Icons** e **React Beautiful DnD**, que oferecem ícones vetoriais modernos, elementos gráficos enriquecidos e funcionalidades de arrastar-e-soltar. Para completar, as bibliotecas **Emotion** e **Shadcn UI** são utilizadas na estilização de componentes com foco em modularidade, reutilização e design moderno, contribuindo para uma experiência visual coesa e de alto nível.

4.1.3.2 Dependências de Desenvolvimento

No grupo das dependências de desenvolvimento, encontram-se ferramentas que otimizam o ambiente de trabalho, asseguram a qualidade do código e facilitam o processo de construção da aplicação. O **Vite**, em conjunto com o plugin **@vitejs/plugin-react**, é utilizado como ferramenta de bundling e desenvolvimento,

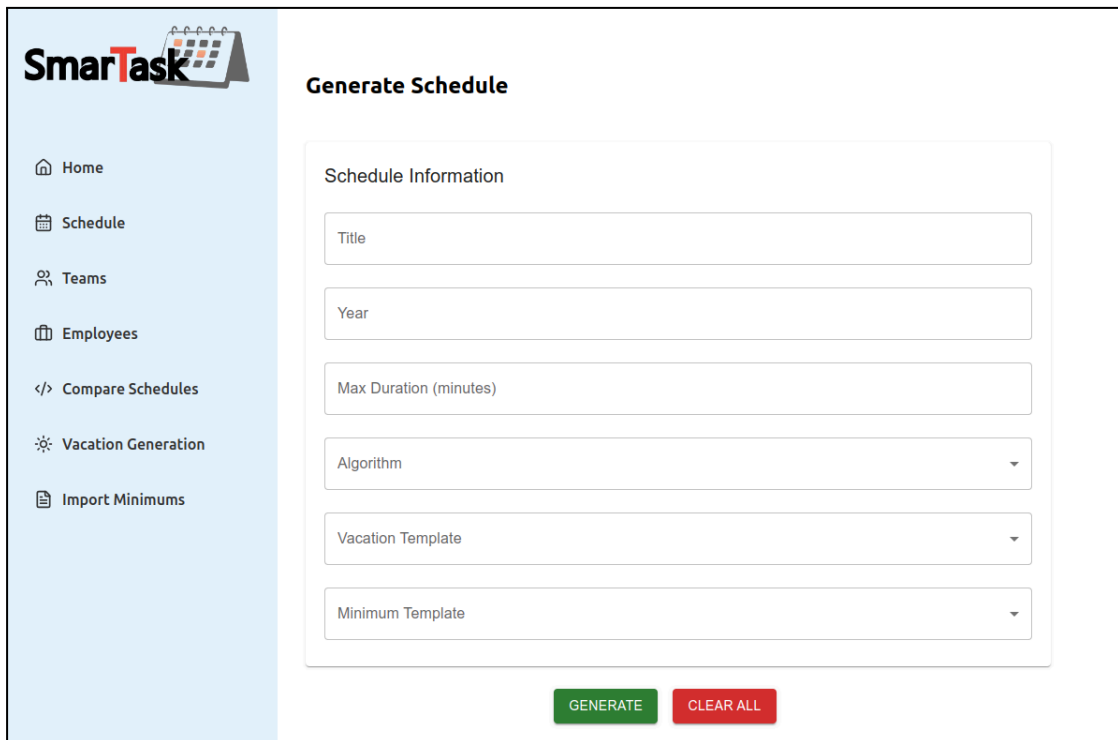
proporcionando inicialização rápida, carregamento instantâneo e build eficiente. A análise estática do código e a padronização das práticas de desenvolvimento são asseguradas pelo **ESLint** e seus plugins associados, que identificam erros e inconsistências de forma automática.

Para garantir compatibilidade entre navegadores e facilitar o uso de estilos modernos, são utilizados o **PostCSS** e o **Autoprefixer**, que processam as folhas de estilo e adicionam os prefixos necessários automaticamente. Por fim, o suporte a tipagem estática e a melhoria da legibilidade do código são promovidos pelo uso dos **tipos TypeScript** dedicados ao React, o que contribui para um desenvolvimento mais seguro e previsível.

4.1.4 Funcionalidades principais do frontend

4.1.4.1 Formulário de criação de horário

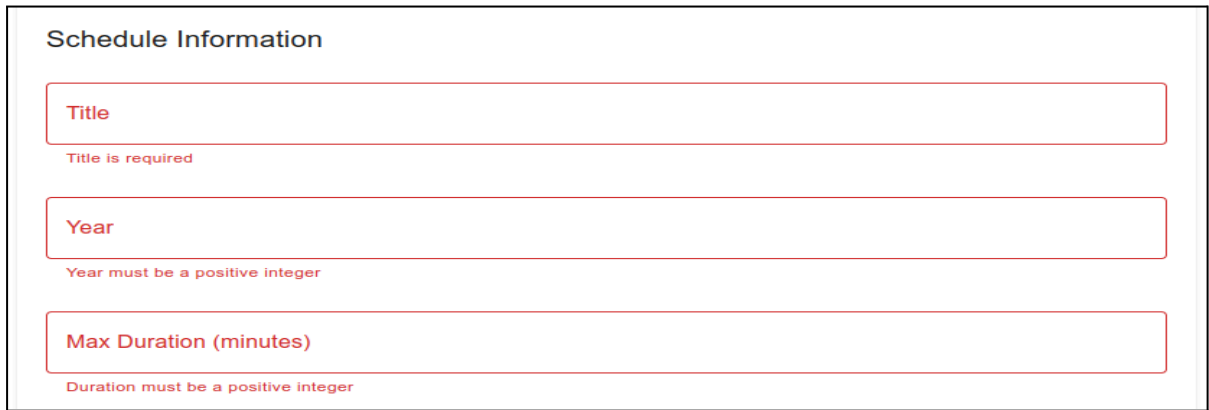
O formulário de geração de horários no sistema inclui validações obrigatórias para garantir a consistência dos dados inseridos. Os campos "Título", "Ano" e "Duração Máxima (minutos)" devem ser preenchidos com valores válidos (texto e inteiros positivos). Além disso, é necessário selecionar um algoritmo de geração, um template de férias e um template de mínimos. Estas validações impedem erros na submissão e asseguram que os dados estão em conformidade com as regras do sistema antes da geração da escala.



The screenshot displays the 'Generate Schedule' interface within the SmarTask application. On the left, a light blue sidebar contains the SmarTask logo and a navigation menu with the following items: Home, Schedule, Teams, Employees, Compare Schedules, Vacation Generation, and Import Minimums. The main content area is titled 'Generate Schedule' and contains a 'Schedule Information' section with several input fields: 'Title', 'Year', 'Max Duration (minutes)', 'Algorithm' (a dropdown menu), 'Vacation Template' (a dropdown menu), and 'Minimum Template' (a dropdown menu). At the bottom of the form, there are two buttons: a green 'GENERATE' button and a red 'CLEAR ALL' button.

Figura 12: Formulário de geração de horários

Exemplo das validações executadas no frontend de forma imediata, proporcionando feedback visual ao utilizador, como indicado pelas caixas vermelhas e mensagens de erro apresentadas na interface.



The image shows a web form titled "Schedule Information". It contains three input fields, each with a red border indicating a validation error. The first field is labeled "Title" and has the error message "Title is required" below it. The second field is labeled "Year" and has the error message "Year must be a positive integer" below it. The third field is labeled "Max Duration (minutes)" and has the error message "Duration must be a positive integer" below it.

Figura 13: Validações no formulário de geração de horários

4.1.4.2 Validação de template de férias em função do número de dias de férias

Durante a importação do template de férias (em formato CSV), o sistema executa validações automáticas para garantir a integridade dos dados. São verificados os seguintes critérios:

- Todos os colaboradores devem ter exatamente **30 dias de férias** atribuídos, independentemente da forma como estão distribuídos;
- Todos os colaboradores existentes na base de dados devem constar no ficheiro;
- O número de colaboradores presentes no template deve coincidir com o número presente na base de dados.

Estas verificações são essenciais para garantir que o template de férias cumpre os critérios definidos e que não existem discrepâncias que possam comprometer o processo de geração dos horários.

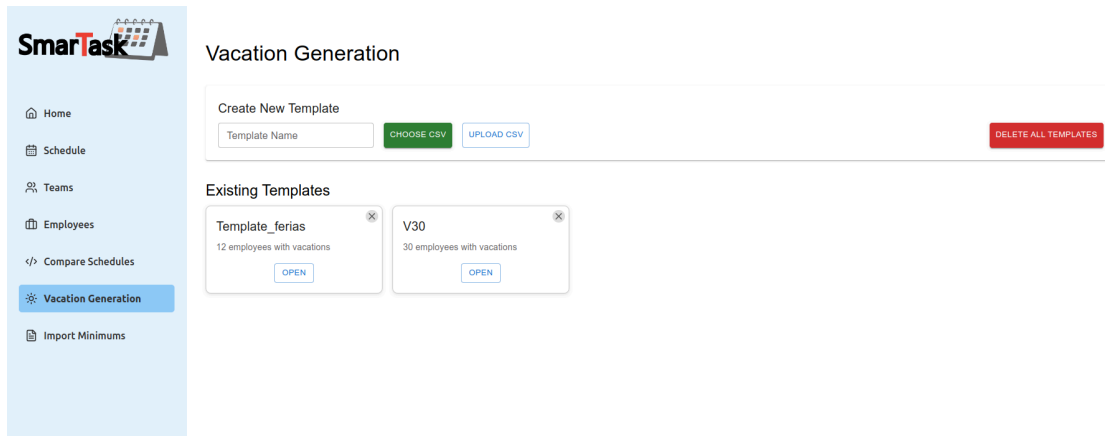


Figura 14: Validações na introdução de um template

4.1.4.3 Validação de Template de Férias com Base nos Empregados Existentes

No processo de criação de um novo horário, o utilizador deve, obrigatoriamente, seleccionar um dos templates de férias previamente registados no sistema. Após a seleção do template, o frontend envia essa informação ao backend, juntamente com a lista de empregados ativos e o ano de referência.

O backend executa então uma validação do ficheiro, verificando:

- Se os dados do template correspondem aos colaboradores registados;
- Se não existem colaboradores em falta;
- Se todas as linhas estão completas e corretamente formatadas.

Caso existam erros, o backend devolve mensagens de erro descritivas, as quais são exibidas de forma clara no frontend, impedindo a continuação do processo até que o ficheiro seja corrigido.

Figura 15:Validação de Template de Férias com Base nos Empregados Existentes

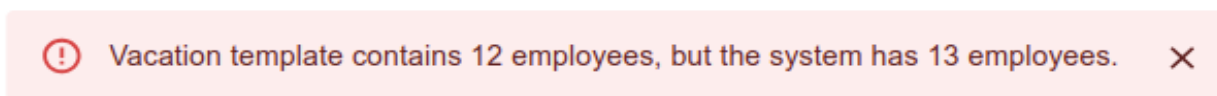


Figura 16: Exemplo de validação

4.1.5 Teste de vulnerabilidades

Durante o desenvolvimento do sistema, foram realizados testes manuais de segurança com o objetivo de identificar vulnerabilidades comuns em aplicações web. Os testes incidiram sobre dois vetores principais:

4.1.5.1 Teste de tentativa de acesso não autorizado

Foi testado o acesso direto à rota protegida **/config**, acessível via **http://localhost:5173/config/**, verificando se havia exposição indevida de informações sensíveis ou configurações internas do sistema. O objetivo foi garantir que apenas utilizadores autenticados ou com permissões adequadas pudessem aceder a essa

página. O resultado do teste confirmou que a rota estava devidamente protegida, não sendo possível aceder sem permissões válidas.(ver Figura 17)

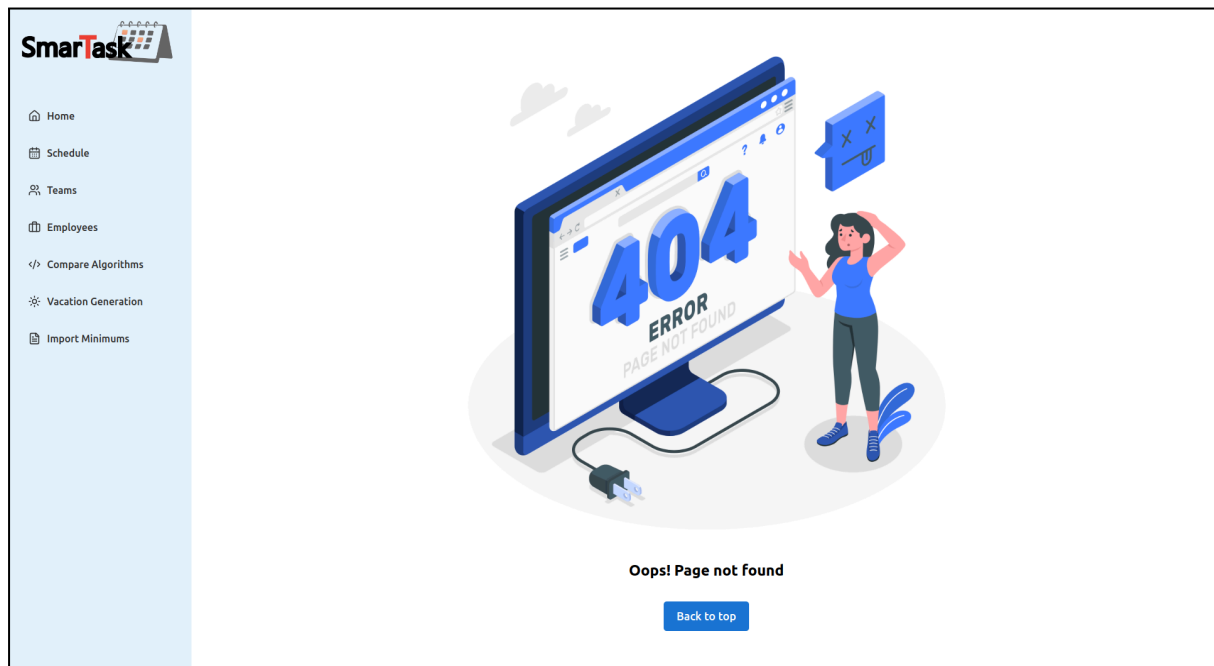


Figura 17: Tentativa não autorizada

4.1.5.2 Teste de Cross-Site Scripting (XSS)

Foi realizado um teste de injeção de código malicioso no frontend através do seguinte payload típico de XSS:

```
<script>alert('XSS')</script>
```

Este código foi introduzido em campos de texto editáveis da aplicação com o intuito de verificar se seria interpretado como HTML ou JavaScript ativo.

No entanto, o sistema tratou o conteúdo inserido como texto normal, exibindo-o exatamente como foi escrito, sem executar o script. Este comportamento confirma que o frontend está protegido contra ataques XSS, através da sanitização ou escape apropriado do conteúdo inserido pelo utilizador.

A imagem abaixo ilustra o resultado do teste:

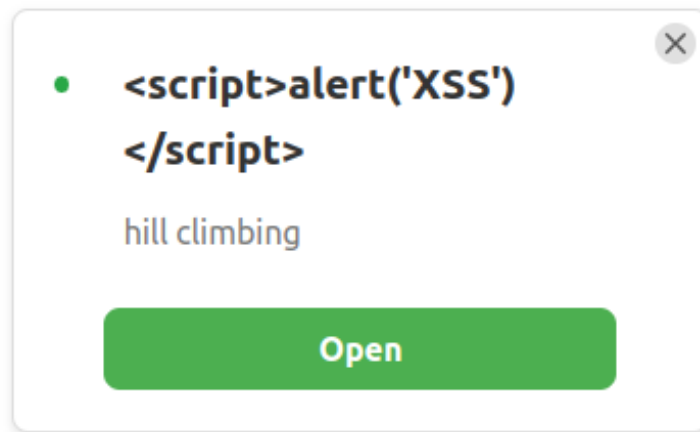


Figura 18: teste de cross-Site Scripting (XSS)

4.1.6 Página principal e funcionalidades

4.1.6.1 Página de visualização de horários gerados

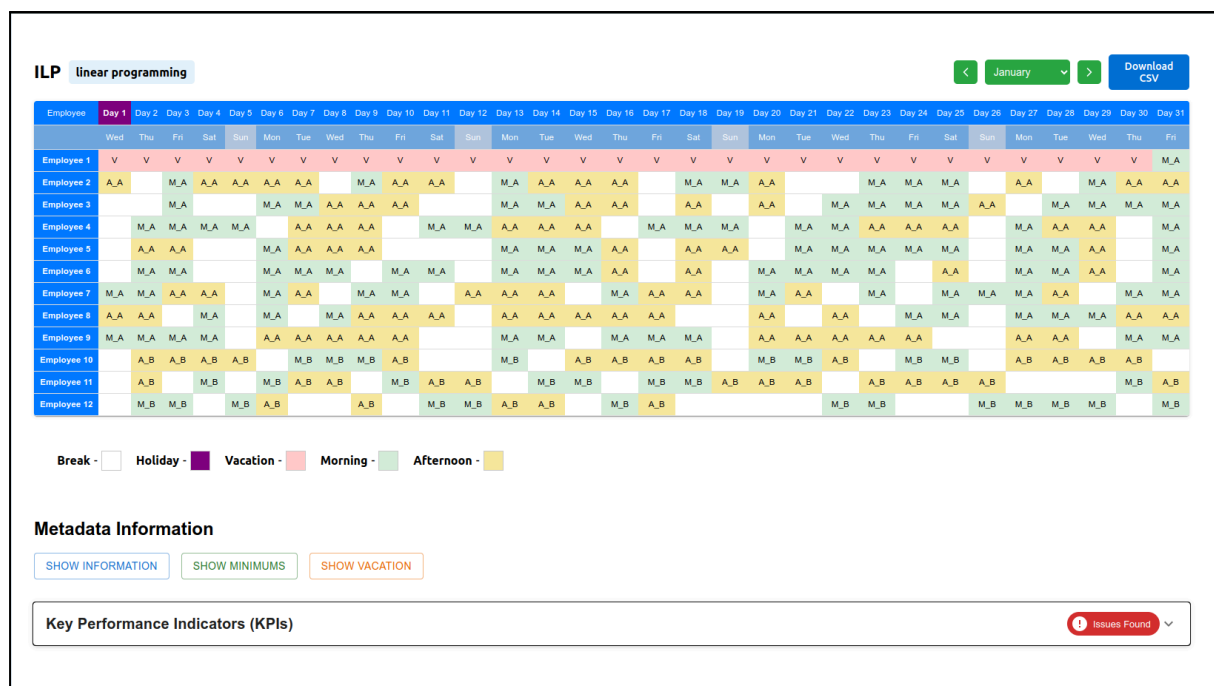


Figura 19: Visualização de Horários Gerados, metadados e KPIS

A página de horários gerados é fundamental para utilizadores e gestores, pois apresenta o resultado final do planeamento, incluindo todos os detalhes relevantes para análise e decisão.

Ela exibe os metadados do horário, como data e parâmetros de geração, garantindo transparência no processo. Além disso, mostra os recursos alocados (“mínimos”) e seus respectivos turnos, facilitando a verificação de conflitos ou sobrecargas.

Também integra o template de férias e outras ausências, assegurando que o planeamento respeite as indisponibilidades da equipa. Os indicadores de desempenho (KPIs) fornecem métricas essenciais para avaliar a qualidade e conformidade do horário gerado.

Por fim, a página permite a exportação dos dados em formato CSV, possibilitando análises adicionais e integração com outras ferramentas de gestão.

4.1.6.2 Home Page, processamento e estado dos horários

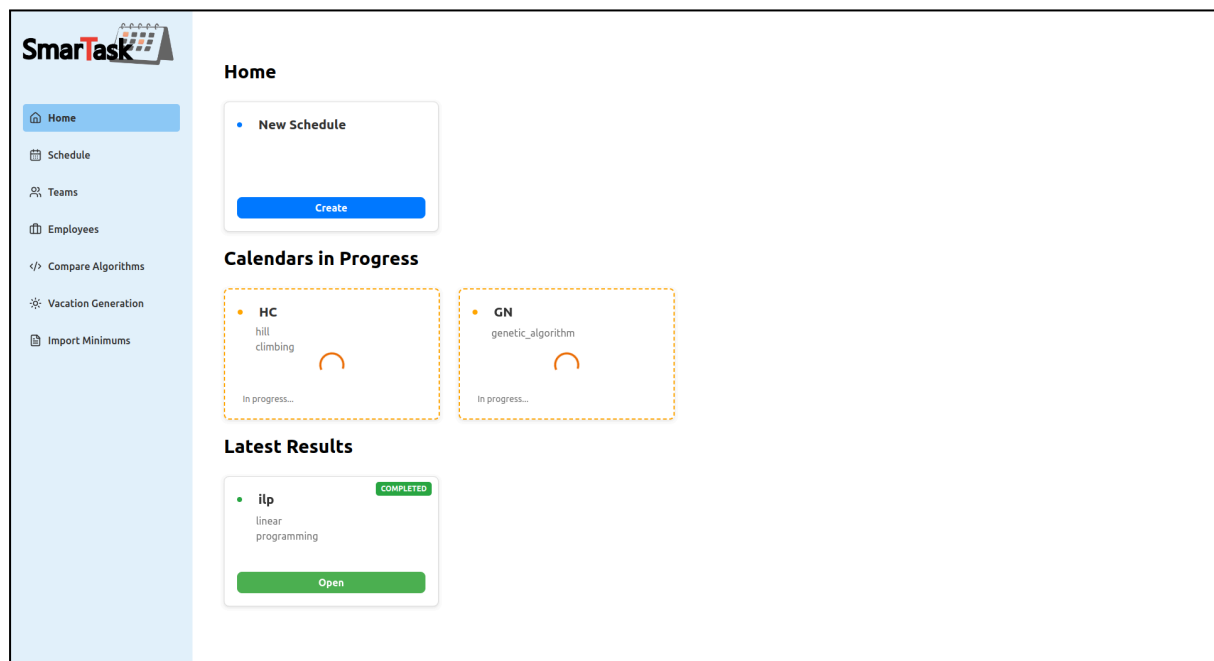


Figura 20: Home page, processamento e estado dos horários

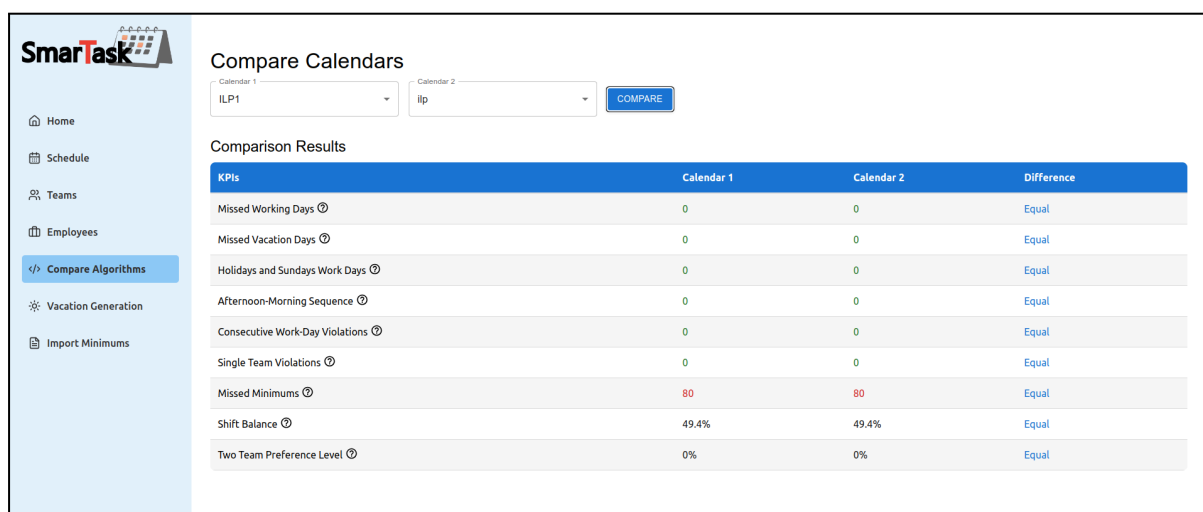
Esta página foi desenvolvida com foco na experiência do utilizador, permitindo-lhe acompanhar de forma clara e eficiente o processo de geração dos horários. Ao iniciar a criação do horário, o utilizador consegue visualizar o estado do processamento em tempo real, confirmando que o calendário está a ser gerado.

A aplicação disponibiliza uma visualização dinâmica e em tempo real dos calendários no seu processo de geração. Esta funcionalidade foi desenvolvida com foco na usabilidade e na comunicação contínua com o utilizador, permitindo o acompanhamento do progresso das tarefas em background de forma responsiva. Para isso, a interface implementa comunicação via WebSocket, utilizando as bibliotecas SockJS e @stomp/stompjs. Assim que uma tarefa de geração de calendário é iniciada, o backend publica notificações de progresso através do canal /topic/comparison/all. O frontend está subscrito a este canal e, ao receber a notificação, atualiza imediatamente o estado da interface, proporcionando uma experiência contínua e informativa.

Após a conclusão, os horários finalizados são automaticamente direcionados para a secção **“Latest Results”**, onde ficam disponíveis os resultados dos últimos horários gerados pelo gestor ou utilizador. Nesta secção, além de consultar os planeamentos recentes, o utilizador pode verificar se o horário foi validado ou “falhou” durante o processo, permitindo um acompanhamento completo do estado de cada execução. Esta funcionalidade garante acesso rápido, atualizado e transparente ao histórico dos horários.

A página integra todas as ferramentas necessárias para o acompanhamento do ciclo completo: desde a criação, passando pela visualização do progresso da geração, até à consulta dos últimos horários gerados. Essa visão unificada oferece uma navegação intuitiva e eficiente.

4.1.6.3 Página de comparação de soluções



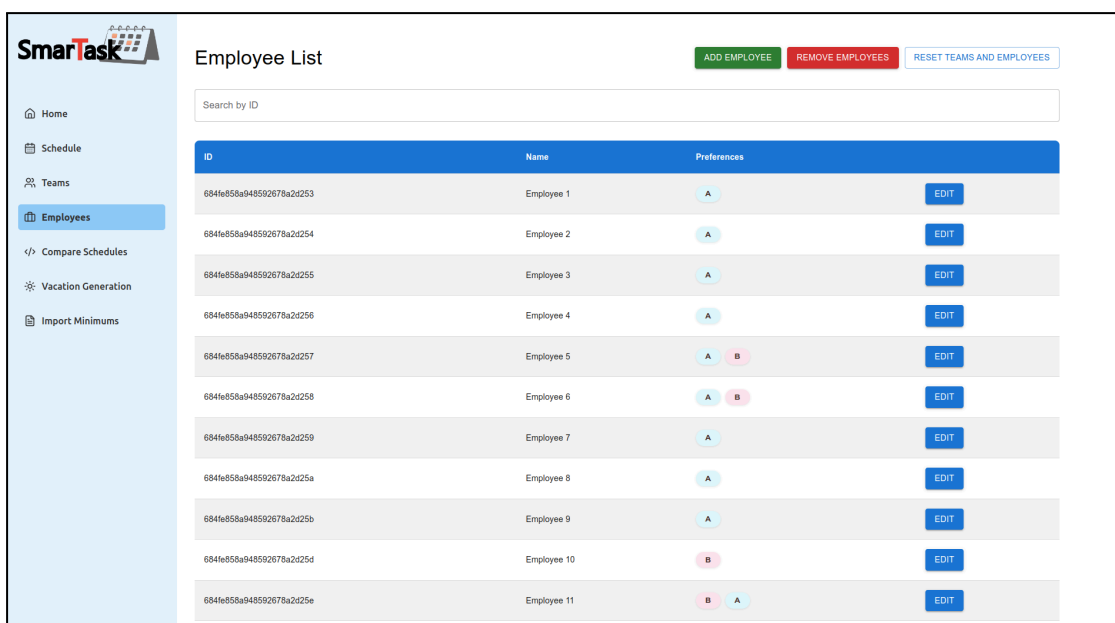
KPIs	Calendar 1	Calendar 2	Difference
Missed Working Days ⓘ	0	0	Equal
Missed Vacation Days ⓘ	0	0	Equal
Holidays and Sundays Work Days ⓘ	0	0	Equal
Afternoon-Morning Sequence ⓘ	0	0	Equal
Consecutive Work-Day Violations ⓘ	0	0	Equal
Single Team Violations ⓘ	0	0	Equal
Missed Minimums ⓘ	80	80	Equal
Shift Balance ⓘ	49.4%	49.4%	Equal
Two Team Preference Level ⓘ	0%	0%	Equal

Figura 21: Comparação de 2 horários gerados

Nesta página está disponível uma das funcionalidades mais relevantes do sistema, voltada especialmente para o apoio à tomada de decisão por parte do gestor. A

funcionalidade de comparação permite analisar, de forma objetiva e detalhada, diferentes horários previamente gerados. O gestor pode selecionar dois calendários e, a partir da comparação, visualizar os critérios e parâmetros aplicados em cada um deles, tais como distribuição de folgas, dias de trabalho consecutivos, variações de turnos, preferências dos funcionários, entre outros. O sistema exibe essas informações lado a lado, destacando as diferenças encontradas e permitindo uma análise precisa dos impactos de cada configuração. Essa comparação minuciosa contribui significativamente para a escolha do horário mais adequado para a realidade da equipa e da organização, otimizando a alocação de recursos humanos e respeitando as diretrizes operacionais e legais. Como resultado, garante-se uma tomada de decisão mais informada, estratégica e eficiente, alinhada com os objetivos da gestão e com as necessidades dos colaboradores.

4.1.6.4 Lista de Empregados e respectivas preferências



ID	Name	Preferences	Edit
684fe858a948592678a2d253	Employee 1	A	EDIT
684fe858a948592678a2d254	Employee 2	A	EDIT
684fe858a948592678a2d255	Employee 3	A	EDIT
684fe858a948592678a2d256	Employee 4	A	EDIT
684fe858a948592678a2d257	Employee 5	A B	EDIT
684fe858a948592678a2d258	Employee 6	A B	EDIT
684fe858a948592678a2d259	Employee 7	A	EDIT
684fe858a948592678a2d25a	Employee 8	A	EDIT
684fe858a948592678a2d25b	Employee 9	A	EDIT
684fe858a948592678a2d25d	Employee 10	B	EDIT
684fe858a948592678a2d25e	Employee 11	B A	EDIT

Figura 22: Lista de Empregados e respectivas preferências

A primeira imagem apresenta a interface da gestão de empregados e suas preferências. Nesta página, é exibida uma tabela contendo os IDs, nomes dos funcionários e as suas preferências de turno ou disponibilidade.

A interface permite realizar as seguintes ações:

- Adicionar um novo empregado com as suas preferências;
- Eliminar um empregado da lista;
- Pesquisar rapidamente por ID, facilitando a localização de registos específicos.

Esta funcionalidade é essencial para manter informações atualizadas e garantir que as preferências individuais sejam consideradas na geração de horários.

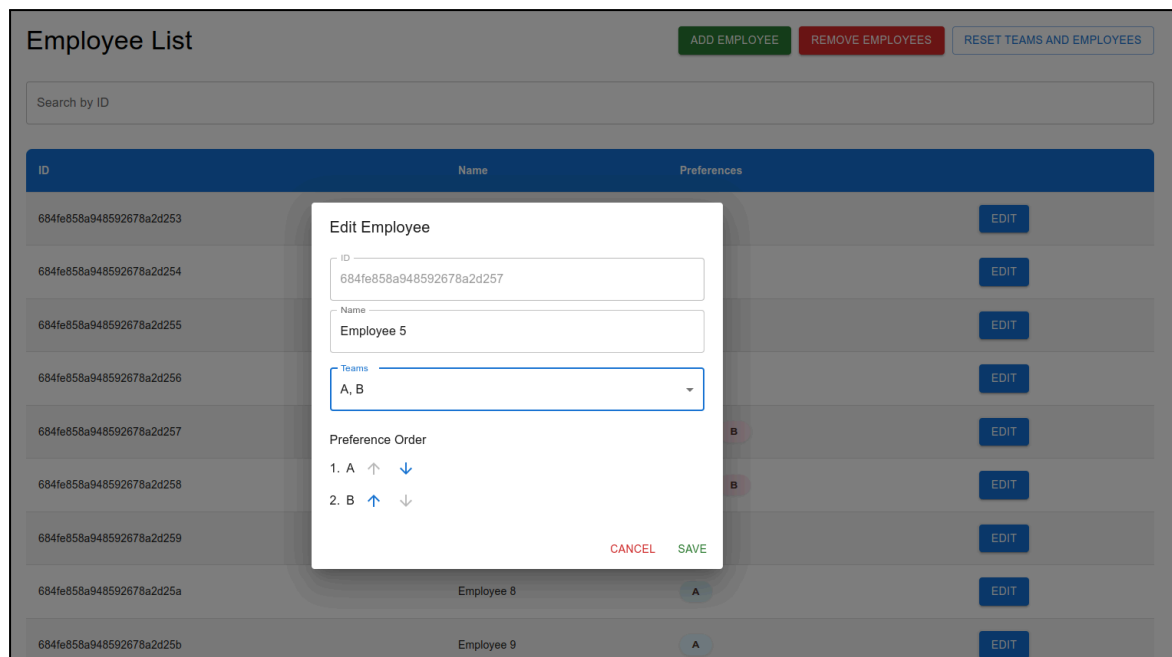


Figura 23: Página de atualização dos empregados

4.1.6.5 Página de listagem de horários

A página de listagem de horários apresenta todos os horários gerados pela aplicação, com base nos dados fornecidos. Nela, é possível visualizar os nomes dos horários gerados e o algoritmo usado nessa geração.

A imagem abaixo ilustra a organização das informações na interface da aplicação:

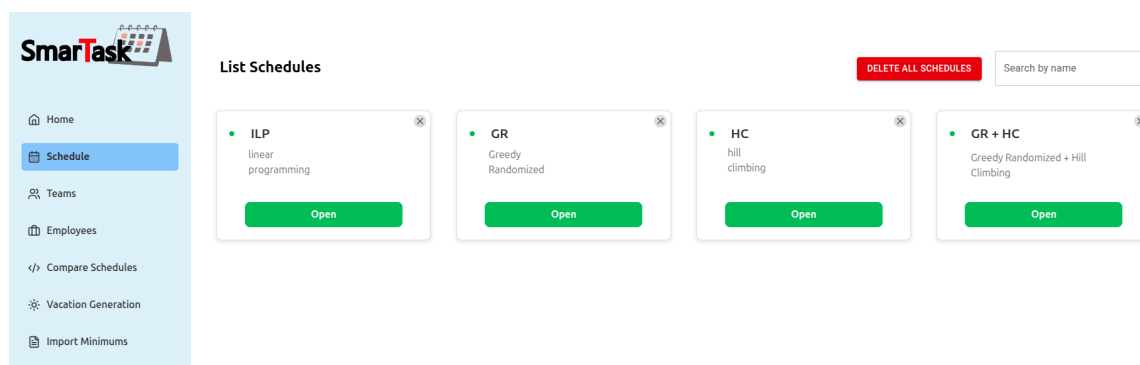


Figura 24: Página da visualização dos horários gerados

4.2 API

O backend do projeto tem o propósito de fornecer uma API REST que disponibiliza um conjunto de funcionalidades associadas à lógica de negócio que variam desde configuração dos dados que serão consumidos pelos algoritmos, solicitação para a geração de uma nova solução e para a visualização de horários gerados e do estado de tarefas (pedidos de geração de horários) e ainda gestão de templates utilizados na geração de horários e gestão da informação de funcionários-equipas (a qual também será consumida pelos algoritmos).

4.2.1 Estrutura da API

A API foi desenvolvida em Java, utilizando o Spring Boot framework, com gestão de dependências através do Maven. A estrutura do desenvolvimento da API segue a estrutura convencional de um projeto Spring Boot orientado a microserviços :

- Models (representam as entidades persistidas na base de dados que também são consumidas no frontend)
- Service (contém a lógica de negócio, incluindo validação dos dados de entrada dos algoritmos)
- Controllers (expõem os endpoints REST, com operações CRUD sobre as entidades)
- Repository (interfaces que fornecem métodos com operações CRUD para cada entidade na base de dados)
- Config (Conjunto de configurações de dados para acesso a base de dados e também para uso do message broker)
- Event (Módulos responsáveis por realizar comunicação assíncrona com os módulos responsáveis pela execução dos algoritmos)

The image shows the Swagger UI for the API. At the top, there's a Swagger logo and a search bar containing "/v3/api-docs". Below this, it says "OpenAPI definition" with a "v0" and "OAS 3.0" badge. A "Servers" section shows a dropdown menu with "http://localhost:8081 - Generated server url". The main section is titled "Team" and "Teams Management". It lists several API endpoints with their methods and descriptions:

- GET** /api/v1/teams/{id} Get a team by ID
- PUT** /api/v1/teams/{id} Update a team
- PUT** /api/v1/teams/{employeeId}/set-team-preference-index/{teamName}/{position} Set team at specific position for an employee
- PUT** /api/v1/teams/{employeeId}/set-first-preference/{teamName} Set first preference team for an employee
- POST** /api/v1/teams/{teamName} Add a new team
- DELETE** /api/v1/teams/{teamName} Delete a team by Name and remove it from all employees
- POST** /api/v1/teams/{teamName}/add-employees Add employees to a team
- GET** /api/v1/teams/name/{name} Get a team by name
- GET** /api/v1/teams/ Get all teams
- DELETE** /api/v1/teams/{teamName}/remove-employee/{employeeId} Remove an employee from a team by name

Figura 25: Endpoints para gestão de equipas e suas preferências para cada funcionário

Clear and Reset Endpoints to clear or reset internal system data for testing or maintenance purposes			^
POST	/clearnreset/reset-employees-teams	Reset employees and teams	v
DELETE	/clearnreset/clean-vacation-templates	Delete all vacation templates	v
DELETE	/clearnreset/clean-schedules	Delete all schedules	v
DELETE	/clearnreset/clean-reference-templates	Delete all reference(minimuns and ideals) templates	v
Schedule Management Endpoints for managing work schedules			^
POST	/schedules/generate	Generate a new schedule	v
POST	/schedules/analyze		v
GET	/schedules/{title}	Get a schedule by title	v
GET	/schedules/fetch	Get all schedules	v
GET	/schedules/fetch/{id}		v
DELETE	/schedules/delete/title/{title}	Delete schedule by title	v
DELETE	/schedules/delete/id/{id}	Delete schedule by ID	v
Employee Employees Management			^

Figura 26: Endpoints para gestão de horários e para limpar documentos e remover alterações às configurações de empregados

Employee Employees Management			^
GET	/api/v1/employees/{id}	Get an employee by ID	▼
PUT	/api/v1/employees/{id}	Update employee's name by their ID	▼
DELETE	/api/v1/employees/{id}	Delete an employee by ID	▼
GET	/api/v1/employees/	Get all employees	▼
POST	/api/v1/employees/	Add a new employee	▼
DELETE	/api/v1/employees/by-name/{name}	Delete an employee by name	▼
reference-controller			^
GET	/reference/{id}	Buscar template de referência por ID	▼
PUT	/reference/{id}	Atualizar template de referência existente por ID	▼
DELETE	/reference/{id}	Excluir template de referência por ID	▼
POST	/reference/create	Criar novo template de referência a partir de CSV	▼
GET	/reference/	Listar todos os templates de referência	▼
DELETE	/reference/by-name/{name}	Excluir template de referência por nome	▼
vacation-controller			^
POST	/vacation/random/{name}		▼
POST	/vacation/csv/{name}		▼
GET	/vacation/		▼
POST	/vacation/		▼

Figura 27: Endpoints para gestão de funcionários, templates de férias e templates de referência (mínimos)

4.2.2 Gestão do estado das tarefas

Uma funcionalidade central do backend é a **gestão do estado das tarefas** de geração de horários, permitindo acompanhar o ciclo de vida de cada pedido submetido.

Cada tarefa (task) passa por um dos seguintes estados:

1. **PENDING** – Pedido criado, à espera de ser processado;
2. **IN_PROGRESS** – O algoritmo iniciou a execução;
3. **COMPLETED** – Horário gerado com sucesso;
4. **FAILED** – O algoritmo falhou ou o tempo máximo foi excedido sem encontrar solução viável.

A atualização do estado é controlada via mensagens trocadas entre os módulos Python e a API, através do RabbitMQ:

1. Ao submeter um pedido, este entra no estado **PENDING**;

2. Quando o módulo Python inicia o algoritmo, informa a API, que atualiza o estado para **IN_PROGRESS**;
3. Se a execução for bem-sucedida, o estado passa para **COMPLETED**;
4. Se ocorrer erro ou expiração do tempo, o estado é atualizado para **FAILED**.

4.2.3 Gestão de templates

A API permite a submissão e validação de templates, usados na geração de um horário, nomeadamente o template de férias (Vacation template) e o template de mínimos e ideais (reference template).

Estes são submetidos como um documento no formato .csv e passados à API, cujo formato e seu conteúdo é previamente validado (baseado em condições de integridade entre o conteúdo do documento e informações presentes na base dados, que devem estar coerentes entre si).

Estes templates são, então, armazenados na base de dados com um título identificador fornecido pelo utilizador, ficando disponíveis para uso posterior pelos algoritmos.

4.2.4 Gestão de funcionários-equipas-preferências

A API oferece suporte para gestão das informações dos funcionários e respectivas preferências:

- Nome;
- ID;
- Equipas associadas;
- Ordem de preferências das equipas associadas;

Estas informações funcionam como um “template” único armazenado na base dados que será acedido pelos módulos python na geração de um novo horário.

4.2.5 Solicitação de pedido de geração de novo horário

A API oferece suporte para submeter pedidos para a geração de novos horários, o que implica enviar um pedido em formato JSON para o message broker para que este seja consumido pelo módulo listener no lado da gestão de algoritmos de execução. O conteúdo desse pedidos inclui:

- Título do horário;
- Nome do algoritmo a ser utilizado;
- Ano de referência;
- Tempo máximo para execução do algoritmo (em minutos);

- Título do template de férias;
- Título do template de mínimos;

4.2.6 Dependências

Em conjunto com o uso do Spring Boot framework, várias dependências foram necessárias para a construção da API :

1. OpenCSV

- `com.opencsv:opencsv:4.1`

2. Spring Boot Starters:

- `org.springframework.boot:spring-boot-starter-amqp`
- `org.springframework.boot:spring-boot-starter-web`
- `org.springframework.boot:spring-boot-starter-data-mongodb`
- `org.springframework.boot:spring-boot-devtools`
- `org.springframework.boot:spring-boot-starter-test`
- `org.springframework.boot:spring-boot-starter-websocket`

3. Spring Data JPA

- `org.springframework.data:spring-data-jpa`

4. Jakarta Persistence API

- `jakarta.persistence:jakarta.persistence-api:3.1.0`

5. Jackson (JSON)

- `com.fasterxml.jackson.core:jackson-databind:2.16.1`

6. Apache Commons CSV

- `org.apache.commons:commons-csv:1.9.0`

7. SpringDoc (OpenAPI/Swagger)

`org.springdoc:springdoc-openapi-starter-webmvc-ui:2.3.0`

8. Lombok

`org.projectlombok:lombok:1.18.34` (*scope: provided*)

9. Spring Rabbit Test

- `org.springframework.amqp:spring-rabbit-test` (*scope: test*)

4.3 Módulos Python

4.3.1 Responsabilidade

Os **módulos Python** constituem um conjunto de scripts e componentes responsáveis pela **execução funcional dos algoritmos de geração de horários** e pela sua integração com o restante sistema.

Estes módulos operam de forma **desacoplada** do backend, comunicando com a API via RabbitMQ, e têm como funções principais:

- Interpretar os pedidos de geração de horários;
- Executar os algoritmos selecionados;
- Processar e armazenar os resultados;
- Realizar análises individuais e comparativas de horários.

4.3.2 RabbitMQClient

Este módulo é responsável por comunicar com o message broker (RabbitMQ). Atua como producer e consumer, utilizando a biblioteca Pika, permitindo a realização de comunicações assíncronas com o protocolo AMQP, usado na comunicação do message broker.

Este módulo também é responsável por lançar os workers necessários para a execução do pedido de um novo horário, sendo este worker, que corre em background, a classe de seleção e execução do algoritmo escolhido, o TaskManager.

Este módulo possui duas fundamentais funcionalidades :

1. Consumir o pedido de geração de um novo horário, recebendo os dados de entrada passados no pedido e lançando um worker TaskManager, que inicia o pedido de geração de uma solução para o algoritmo identificado no pedido. O worker, ao iniciar o processo de busca por uma solução, envia uma mensagem para o message broker, informando o início da execução do algoritmo.
2. No caso do TaskManager informar que o algoritmo encontrou uma solução ou não, é enviada uma mensagem informativa para o message broker. No caso de encontrar uma solução, o RabbitMQClient envia ao MongoDBClient o conteúdo do novo horário gerado, em conjunto com os metadados com informações úteis para serem armazenadas em conjunto com o novo horário na base de dados.

4.3.3 MongoDBClient

Este módulo fornece os métodos para **interagir com a base de dados MongoDB**. Utiliza a biblioteca pymongo e suporta operações de leitura e escrita.

Responsabilidades:

- **Ler templates** de férias e mínimos com base nos títulos definidos;
- **Obter dados** de colaboradores, equipas e preferências;
- **Armazenar os horários gerados**, juntamente com os metadados (ex: algoritmo, data, duração da execução, KPIs calculados, etc.).

4.3.4 TaskManager

Este módulo é responsável por registrar os métodos usados para a geração de um novo horário e gerir a sua execução. Ele recebe o pedido de geração, identificando o nome do algoritmo a ser utilizado, e passa os dados fundamentais necessários para a construção do horário, sendo esses dados:

- a. Vacation template
- b. Mínimos template
- c. Informação dos funcionários-equipas-preferências
- d. Tempo máximo de execução

Todos esses dados são recebidos através de pedidos enviados ao RabbitMQClient e, então, passados ao TaskManager para identificar o tipo de algoritmo a ser usado e iniciar o processo de construção de uma solução. Sendo assim, o resultado da execução do TaskManager é retornado ao RabbitMQClient, independentemente da presença de uma solução (Tarefa completa) ou da não(Tarefa falhada).

4.3.5 Analyser

Este último módulo é o responsável por gerir os pedidos feitos para análise de algoritmos. No caso de um horário ser gerado, aquando da sua visualização, é possível ver a análise feita ao mesmo de forma automática, sem um pedido expresso do utilizador. Caso o pretendido seja obter uma comparação entre dois resultados diferentes, é necessário que o utilizador especifique esses mesmos resultados, através da interface gráfica, para posteriormente poder obter essa comparação.

Este módulo obtém os pedidos provenientes do mesmo endpoint, qualquer que seja o objetivo pretendido (verificação ou comparação). Dependendo do número de ficheiros que sejam passados no pedido, o módulo faz uma chamada para obter uma verificação de um resultado ou uma comparação entre dois resultados. Em ambos os casos, são passadas informações relativas a cada uma das soluções: mapa de férias, mapa de mínimos e ideais, empregados e o ano. Isto é feito, de modo a permitir uma análise justa, mesmo que comparando resultados com diferentes parâmetros (por exemplo, o caso de um horário executado para 2025 e outro para 2026).

Em relação à análise feita, são analisadas um conjunto de métricas, que permitem auxiliar o utilizador a identificar soluções mais e menos rentáveis, entre as quais:

- **Número de Dias de Trabalho Falhados:** cada funcionário é obrigado a trabalhar 223 dias num ano; assim, este número é uma soma do número de dias falhados para todos os funcionários (caso um funcionário trabalhe três dias em excesso, esta variável é incrementada em 3 e não apenas 1);

- **Número de Dias de Férias Falhados:** cada funcionário tem direito a 30 dias de férias num ano; esta métrica é uma soma do número de dias de férias falhados para todos os funcionários;
- **Número de Dias de Trabalho em Feriados e Domingos:** feriados e domingos são tidos em conta como dias especiais e, por isso, cada funcionário não pode trabalhar mais do que 22 vezes nesses dias;
- **Número de Turno Tarde Manhã Consecutivos:** é estritamente proibido que um funcionário trabalhe num turno da tarde num dado dia e que faça o turno da manhã no dia exatamente a seguir; esta métrica faz uma soma do número de vezes que esta situação ocorre;
- **Dias Consecutivos de Trabalho:** cada funcionário não pode trabalhar mais do que 5 dias consecutivos; assim sendo, esta métrica faz uma contagem do número de vezes que isso ocorre para o dado horário;
- **Violações para Equipas Únicas:** esta métrica verifica se cada funcionário pertencente a uma única equipa trabalha, de facto, para essa equipa e apenas essa equipa;
- **Número de Mínimos Falhados:** como mencionado, para cada dia, turno e equipa existe um número mínimo associado de trabalhadores necessários; esta métrica faz a contagem do número de vezes que o mesmo é falhado;
- **Relação entre Turnos:** uma das formas de garantir um horário justo e equilibrado é assegurar que não existem funcionários com muitos turnos de um só tipo, gerando desbalanceamentos; por isso, esta métrica representa o funcionário com pior distribuição de turnos, representando esse número numa percentagem (por exemplo, 30% demonstra que o funcionário com pior distribuição de turnos trabalha 70% dos seus turnos tarde/manhã e apenas 30% do oposto);
- **Preferências entre Equipas:** esta métrica faz uma média da percentagem de turnos feitos para a equipa preferida, tendo em conta apenas os funcionários que trabalham para mais de uma equipa; por isso, também esta métrica é apresentada no formato de percentagem;

4.4 Algoritmos

4.4.1 Responsabilidade

Esta secção descreve os algoritmos de geração de horários de trabalho utilizados ou desenvolvidos ao longo do projeto. Incluem-se tanto algoritmos que demonstraram bons resultados — em termos de qualidade das soluções e desempenho — como algoritmos que apresentaram limitações, quer em tempo de execução, quer em escalabilidade.

Os algoritmos foram avaliados com base na:

- **Qualidade das soluções:** capacidade de cumprir as restrições laborais e operacionais;
- **Eficiência algorítmica:** tempo necessário para gerar uma solução viável;
- **Escalabilidade:** capacidade de manter bom desempenho com aumento de dados e restrições.

4.4.2 Dados de entrada

Todos os algoritmos recebem como entrada os mesmos **quatro conjuntos de dados**, fundamentais para construir uma solução:

- **Template de férias**, contendo o mapeamento anual dos 30 dias de férias para cada funcionário;
- **Template de mínimos**, contendo um mapa do número mínimo de funcionários que deve trabalhar em cada turno de cada equipa, para os 365 dias do ano;
- **Informação dos empregados**, incluindo equipa(s) a que cada empregado pertence e sua ordem de preferência;
- **Tempo máximo de execução** dado para cada algoritmo procurar uma solução.

Cada um desses dados são recebidos pelos algoritmos e são processados e utilizados de forma distinta por cada algoritmo, mas que no final são igualmente utilizados para a geração de uma solução.

Estes dados de entrada são armazenados nos metadados de cada horário gerado, algo que permite ao utilizador consultá-los caso necessite para tomar decisões ao realizar novos pedidos de geração de horários.

4.4.3 Restrições

Para além dos dados de entrada, os algoritmos devem respeitar um conjunto fixo de **regras laborais** que funcionam como restrições fundamentais. O cumprimento destas restrições contribui diretamente para os KPIs de qualidade:

- Exatamente **223 dias de trabalho por ano por funcionário**;
- No máximo **22 domingos/feriados trabalhados**;
- No máximo **5 dias consecutivos de trabalho**;
- **Proibição de transições T→M** em dias consecutivos;
- **Imposição de folgas nos dias de férias definidos**;

4.4.4 Algoritmos com bons resultados

Abaixo segue a explicação dos 4 algoritmos que foram explorados ao longo do projeto e que tiveram resultados aceitáveis, apesar de existir ainda espaço para melhorias qualitativas nos mesmos. De forma sumarizada, inclui-se nessa lista algoritmos de natureza iterativa, com uso de técnicas para encontrar as melhores soluções(horários) utilizando várias técnicas, especialmente heurísticas, para calcular melhorias e trocas locais (com exceção do algoritmo ILP). Com exceção do algoritmo de programação linear inteira, que utiliza modelos matemáticos e que gera soluções com métodos e resultados determinísticos, todos os algoritmos desta secção utilizam estratégias de atribuição inicial de horário aleatória e, após essa etapa, realizam melhorias incrementais sucessivas até encontrar a melhor solução.

4.4.4.1 Programação Linear Inteira (ILP - Integer Linear Programming)

Programação Linear Inteira (ILP) é uma técnica de otimização matemática usada para resolver problemas nos quais se busca maximizar ou minimizar uma função linear, sujeita a um conjunto de restrições lineares, com a condição de que algumas ou todas as variáveis de decisão devem assumir **valores inteiros**. É amplamente utilizada em problemas de alocação de recursos, escalonamento e planeamento onde soluções fracionárias não fazem sentido.

Neste projeto, o algoritmo ILP foi desenvolvido com suporte da biblioteca Pulp, conhecida por fornecer toolbox com ferramentas para definir modelos ILP, definir suas variáveis e restrições de forma prática, representado restrições fundamentais e restrições de penalização como expressões matemáticas.

Variáveis de decisão principais

- $x[f][d][t]$ (binária): indica se o funcionário f está escalado no dia d para o turno t ($0 = \text{folga}$, $1 = \text{manhã}$, $2 = \text{tarde}$).
- $y[d][t][e]$ (inteira): número de membros da equipa e escalados no turno t no dia d .

Variáveis de decisão auxiliares

- $\text{penal}[d][t][e]$: variáveis contínuas não-negativas que representam a **diferença negativa entre o número mínimo exigido** de funcionários no turno e e o número efetivamente escalado. São usadas na função objetivo como penalizações.

A **função objetivo** do modelo tem como finalidade **minimizar o total de incumprimentos das metas mínimas de cobertura de turnos por equipa ao longo do ano**.

Mais especificamente, para cada dia do ano, cada turno (manhã e tarde) e cada equipe, existe um valor mínimo desejado de funcionários escalados. A função objetiva avaliar o quanto cada uma dessas metas não é atingida — isto é, a **diferença entre o número mínimo necessário e o número real de funcionários escalados** — e **acumula essas falhas** como penalizações.

Essas penalizações são representadas por variáveis auxiliares contínuas, que são incluídas na função objetivo como uma soma ponderada (ou simples) de desvios. O modelo trabalha, portanto, para encontrar uma escala que **reduza ao máximo essas falhas de cobertura**, garantindo que o maior número possível de metas de cobertura mínima seja cumprido, respeitando todas as demais restrições obrigatórias do sistema.

As **restrições** do modelo são adicionadas de forma programática por meio de laços que percorrem os conjuntos de funcionários, dias do ano e turnos. Cada restrição é expressa como uma **condição linear envolvendo as variáveis de decisão** e é incluída no modelo utilizando a interface de modelagem do solver.

Para cada tipo de regra definida (como limites de trabalho, folgas, transições, entre outros), são criadas **expressões matemáticas** que representam a lógica da restrição, utilizando as variáveis binárias associadas à escala de trabalho. Essas expressões são **acopladas ao modelo como desigualdades ou igualdades**.

As restrições são aplicadas individualmente ou em blocos, dependendo da lógica, e são identificadas por **nomes únicos** que ajudam a rastrear a sua origem e interpretação. Além disso, algumas restrições são aplicadas globalmente (como exigências mínimas por dia), enquanto outras são específicas por funcionário ou por período contínuo de dias.

Dessa forma, o modelo é alimentado com **um conjunto completo de restrições formais**, que limitam o espaço de soluções viáveis e garantem que apenas escalas compatíveis com todas as regras sejam consideradas pelo solucionador durante a otimização.

Como o modelo encontra a solução ótima

1. **Inicialização dos dados:** lê as férias, estrutura das equipes, metas de cobertura mínima e define os dias do ano.
2. **Criação do modelo ILP:** define as variáveis e insere todas as restrições no modelo.
3. **Definição da função objetivo:** configura o modelo para minimizar o total de falhas na cobertura mínima.
4. **Resolução com solver:** utiliza o solver PULP_CBC_CMD com limite de tempo e tolerância de gap relativa.

4.4.4.2 Hill Climbing

O algoritmo baseia-se no princípio do *Hill Climbing*, uma técnica de busca local iterativa que parte de uma solução inicial (gerada aleatoriamente) e procura melhorar incrementalmente essa solução ao minimizar penalizações associadas a violações de critérios definidos. A mesma não garante encontrar a solução ótima global, mas permite alcançar soluções satisfatórias com alta eficiência computacional, especialmente útil em problemas de grande escala.

A solução inicial é gerada respeitando algumas restrições:

- Os turnos são atribuídos apenas em dias em que o trabalhador **não está de férias**.
- São evitadas transições inválidas entre turnos, como trabalhar à tarde em um dia e pela manhã no dia seguinte ($T \rightarrow M$).
- As preferências de equipe (Equipa A, Equipa B ou ambas) são consideradas, direcionando o tipo de turno que cada colaborador pode assumir.

A qualidade de cada escala gerada é medida por um vetor de penalizações provenientes dos seguintes critérios:

- **Critério 1 (C1):** Número de ocorrências em que um colaborador trabalha 5 dias consecutivos — o limite máximo permitido.
- **Critério 2 (C2):** Excesso de dias trabalhados em finais de semana ou feriados (máximo de 22 por ano).
- **Critério 3 (C3):** Dias com menos trabalhadores do que o mínimo exigido.
- **Critério 4 (C4):** Excesso de folgas além do limite anual.
- **Critério 5 (C5):** Violações da regra de transição de turnos inválida ($T \rightarrow M$).
- **Critério 6 (C6):** Violação de mínimos por turno e por equipa, comparando a escala gerada com os mínimos diários definidos externamente (via CSV).

Esses critérios são avaliados conjuntamente para orientar a busca por uma solução melhor, sendo que a soma ponderada das penalizações atua como a função objetivo do algoritmo.

A partir da solução inicial, pequenas modificações são aplicadas iterativamente, como a troca de turnos entre dois dias ou entre dois colaboradores. Cada modificação é avaliada com base nos critérios descritos. Se uma nova versão do calendário apresenta uma penalização total menor, ela substitui a solução atual. O processo continua até que nenhuma melhoria adicional seja encontrada após um determinado número de iterações.

4.4.4.3 Greedy Randomized

O objetivo do algoritmo é construir, de forma progressiva e eficiente, uma atribuição de turnos para todos os funcionários, respeitando múltiplas restrições operacionais e procurando cumprir com os valores mínimos e ideais definidos, por turno e por equipa.

A lógica central do algoritmo baseia-se nos seguintes princípios:

Construção iterativa: A cada iteração, o algoritmo seleciona um funcionário elegível (com menos de 223 dias atribuídos) e tenta alocar-lhe um novo turno (manhã ou tarde) em um dos dias disponíveis, excluindo os dias de férias previamente definidos.

Critério de viabilidade (função f_1): Antes de realizar qualquer atribuição, o algoritmo verifica se a atribuição respeita restrições obrigatórias, tais como:

- Máximo de **5 dias consecutivos de trabalho**;
- Máximo de **22 domingos e feriados trabalhados por funcionário**;
- Proibição de transições de turno inválidas (tarde → manhã).

Critério de custo (função f_2): Definidas todas as atribuições válidas, o algoritmo avalia o impacto da escolha sobre a cobertura de turnos da equipa. A função de custo penaliza atribuições que estejam abaixo do mínimo necessário, valoriza aquelas que caminham em direção à cobertura ideal, e penaliza excessos além do ideal.

Aleatoriedade controlada: Em vez de seguir rigidamente a melhor escolha a cada passo (como num algoritmo puramente ganancioso), o algoritmo realiza múltiplas tentativas aleatórias por iteração (definidas pelo parâmetro **num_iter = 10**). Isso permite escapar de escolhas locais subótimas e explorar diferentes regiões da solução. Para este caso, foram estipuladas 10 iterações como um número ótimo, para a resolução do problema base.

```

1.  Table  $\leftarrow$  an empty timetable
2.  While Table not complete do
3.      P  $\leftarrow$  set of persons in  $P_1$  without a complete timetable
4.      If P is empty do
5.          P  $\leftarrow$  set of persons in  $P_2$  without a complete timetable
6.      EndIf
7.      p  $\leftarrow$  a person randomly selected from P
8.      f_value  $\leftarrow$  +inf
9.      count  $\leftarrow$  0
10.     While f_value > 0 and count < numIter do
11.         d  $\leftarrow$  a day randomly selected from the possible working days still not assigned to person p
12.         s  $\leftarrow$  a randomly selected shift
13.         If  $f_1(\text{Table}, p, d, s)$  is TRUE do
14.             count  $\leftarrow$  count + 1
15.             For  $t \in T_p$  do
16.                 f_aux  $\leftarrow$   $f_2(\text{Table}, d, s, t)$ 
17.                 If f_aux < f_value do
18.                     f_value  $\leftarrow$  f_aux
19.                      $(p, d, s, t)_{best} \leftarrow (p, d, s, t)$ 
20.                 EndIf
21.             EndFor
22.         EndIf
23.     EndWhile
24.     Table  $\leftarrow$  Table +  $(p, d, s, t)_{best}$ 
25. EndWhile

```

Figura 28: Pseudo-Código Relativo ao Algoritmo

4.4.4.4 Greedy Randomized com Refinamento via Hill Climbing

Este algoritmo é uma solução híbrida entre dois outros métodos já apresentados. A construção do horário é feita em duas fases principais:

Construção da Solução por Heurística Greedy Randomized : Nesta etapa, o algoritmo preenche progressivamente o horário de cada funcionário com base em escolhas pseudo-aleatórias, porém guiadas por critérios de viabilidade e qualidade:

- A cada iteração, um funcionário elegível (com menos de 223 dias atribuídos) é selecionado aleatoriamente.
- Para esse funcionário, avalia-se uma quantidade limitada de opções aleatórias de dia e turno (10 tentativas), aplicando filtros de viabilidade:
 - **Restrição de até 5 dias consecutivos de trabalho;**
 - **Limite de 22 domingos/feriados trabalhados;**
 - **Proibição de transições de turno tarde para manhã.**

- As opções viáveis são avaliadas segundo uma função de custo (baseada em quão próximo a atribuição está dos valores mínimos/ideais) e a melhor é escolhida.
- O processo é repetido até que todos os funcionários tenham 223 dias de trabalho atribuídos ou não haja mais opções válidas.

Otimização por Hill Climbing : Uma vez construída a solução inicial, o algoritmo entra em uma fase de otimização local com base em *Hill Climbing*, onde pequenas modificações são feitas iterativamente para reduzir um score de penalização:

- A cada iteração, são trocadas atribuições de dois dias distintos para um funcionário aleatoriamente escolhido.
- A troca só é realizada se não violar restrições obrigatórias (ex.: transições proibidas ou trabalho em férias).
- A nova escala é avaliada com base em uma função de custo composta por cinco critérios:
 1. Violação de dias consecutivos acima do permitido;
 2. Excesso de domingos e feriados trabalhados;
 3. Falta de cobertura mínima por turno/equipe;
 4. Desvio da carga de trabalho ideal (223 dias);
 5. Transições inválidas entre turnos (tarde para manhã).
- A modificação é aceite se reduzir o score total. Caso contrário, é descartada.
- O processo é repetido até atingir um número máximo de iterações ou uma fase de estagnação (sem melhorias).

Esta abordagem permite combinar os excelentes resultados obtidos pelo algoritmo Greedy Randomized (especialmente em comparação com a solução obtida de forma aleatória pelo Hill Climbing), fazendo posteriores melhorias a esses mesmos resultados, usando o Hill Climbing.

4.4.5 Algoritmos com resultados insatisfatórios

Abaixo segue-se a explicação dos dois algoritmos que foram explorados no desenvolvimento do projeto e que, apesar de serem algoritmos conhecidos no campo da otimização e inteligência artificial, não apresentaram, ao longo do seu desenvolvimento, bons resultados.

Para o caso do algoritmo genético, num dado momento do seu desenvolvimento, o mesmo deixou de encontrar soluções satisfatórias, em especial ao tentar cumprir com as restrições laborais básicas, como também ao cobrir os mínimos diários de funcionários por turnos e por equipes.

Para o caso do algoritmo de Satisfação de Restrições, apesar de gerar bons resultados para teste de apenas 1 mês (30 dias), o mesmo, ao procurar por uma solução completa (365 dias), era incapaz de realizar esse tipo de tarefas, uma vez que as restrições

impostas obrigavam a um elevadíssimo número de combinações, o que revelou-se computacionalmente insuportável.

4.4.5.1 Algoritmo Genético

Algoritmo que implementa uma abordagem baseada em algoritmos genéticos. O algoritmo utiliza a biblioteca **DEAP** (Distributed Evolutionary Algorithms in Python) para modelar e resolver o problema como um processo evolutivo. A representação de cada indivíduo da população é uma matriz de dimensão (12 x 365), onde cada elemento indica o turno atribuído ao funcionário em um determinado dia: 0 (folga), 1 (manhã), 2 (tarde) ou “F” (férias).

1. Inicialização da população

Cada indivíduo é gerado respeitando regras rígidas, como número de dias de trabalho, distribuição de férias, controle de trabalho em feriados e sequências válidas de turnos.

2. Avaliação (Função Objetivo)

A função de avaliação penaliza desvios em relação às seguintes restrições:

- Número incorreto de dias de trabalho;
- Excesso de domingos/feriados trabalhados;
- Mais de cinco dias consecutivos de trabalho;
- Ocorrência de transições inválidas ($T \rightarrow M$).

A penalização é composta e ponderada, de forma a priorizar as violações mais críticas.

- **Mutação:** troca de dois dias de turno aleatórios de um funcionário, respeitando as restrições fundamentais.
- **Cruzamento:** troca parcial da escala de um funcionário entre dois indivíduos.
- **Seleção:** usa torneios para selecionar os melhores individuais, mantendo elite (os 5 melhores) a cada geração.

3. Evolução

A população evolui por 120 gerações, com parâmetros de cruzamento (CXPB = 0.7) e mutação (MUTPB = 0.3). A cada geração, os indivíduos são avaliados, selecionados e cruzados, e o melhor indivíduo é monitorado.

4.4.5.2 Problema de Satisfação de Restrições (CSP – Constraint Satisfaction Problem)

O núcleo do sistema é baseado em um modelo de **Problema de Satisfação de Restrições (CSP – Constraint Satisfaction Problem)**, uma técnica amplamente utilizada para resolver problemas combinatórios complexos.

Estrutura Geral

Cada variável do problema representa o turno atribuído a um funcionário em um determinado dia (**E1_1**, **E1_2**, ..., **E12_30**). Os valores possíveis das variáveis incluem turnos da manhã (**M_A**, **M_B**), da tarde (**T_A**, **T_B**), folga (**0**) ou férias (**F**), com o domínio de cada variável sendo condicionado pela equipa à qual o funcionário pertence e pelos seus dias de férias.

As restrições laborais já explicitadas são implementadas como funções de verificação num grafo de restrições do CSP e que são avaliadas constantemente durante o processo de atribuição de valores às variáveis.

Estratégia de Resolução : O algoritmo utiliza uma **busca com propagação de restrições**. A cada passo, ele seleciona a variável com o menor número de valores possíveis (heurística de domínio mínimo), tenta atribuir-lhe um valor válido e propaga os efeitos dessa decisão reduzindo os domínios das variáveis restantes. A busca é limitada por um tempo máximo de execução (timeout configurável).

Além disso, a aleatoriedade na escolha de valores e a persistência de férias entre execuções fornecem variabilidade e reprodutibilidade no processo.

4.5 Message Broker

Com o objetivo de obter uma comunicação assíncrona entre a API e os módulos python, foi configurado um message broker utilizando RabbitMQ, que implementou o protocolo "Advanced Messaging Queueing Protocol" (AMQP), que permite que ambos os componentes da arquitetura comuniquem e partilhem mensagens entre si usando channels e queues para realizar duas operações fundamentais:

- Realizar o pedido de geração de um novo horário
 - API -> mensagem de pedido -> RabbitMQClient (python modules)
- Atualizar o estado de execução do processo de geração de um horário
 - RabbitMQClient (python modules) -> mensagem atualização de status -> API

Este componente é instanciado na aplicação por meio da definição do seu service na composição do docker-compose.yml do projeto, enquanto que a criação das necessárias queues e channels para um pedido de geração de um novo horário e atualização do estado do pedido (task status), é executado pelos componentes API dos módulos python (especificamente o RabbitMQClient). Após ambos estabelecerem conexão com o message broker, conectam-se ao serviço de acesso a base dados, dentro da mesma network criada para comunicação dos componentes/containers na composição dos serviços.

4.6 Base de Dados

Para este projeto, é necessária a capacidade de armazenar vários conjuntos de dados, nomeadamente:

- Informações sobre colaboradores, equipas e preferências;
- Templates de férias e mínimos;
- Horários gerados, juntamente com os respetivos metadados que indicam os dados de entrada usados na sua geração;
- Tarefas (*tasks*) com o estado da execução dos algoritmos de geração de horários.

Assim, foi configurada uma **base de dados NoSQL – MongoDB**, responsável por armazenar todas estas entidades sob a forma de **documentos individuais agrupados em coleções**. Estes dados são acedidos tanto pela **API**, que permite a visualização e atualização de alguns destes conteúdos, como pelo módulo MongoDBClient, que lê os dados dos templates e colaboradores e, após a geração de um novo horário, insere o novo calendário na coleção correspondente, juntamente com os seus metadados.

Este componente (MongoDB) é instanciado na aplicação através da definição do seu serviço no ficheiro docker-compose.yml. Os restantes componentes que necessitam de acesso persistente aos dados — nomeadamente a **API** e os **módulos Python** (em especial o MongoDBClient) — estabelecem ligação ao serviço de base de dados dentro da mesma **rede Docker** definida para permitir a comunicação entre os vários componentes/containers do sistema.

4.7 Composição da aplicação

Para compor e disponibilizar toda a aplicação de forma funcional, foi utilizada a ferramenta de orquestração **Docker Compose**, baseada na tecnologia de containerização **Docker**. Esta ferramenta permite a definição e execução de um conjunto de serviços interdependentes — **Frontend, API, message broker, módulos Python e base de dados** — que são automaticamente instanciados como **containers Docker**.

Todos estes serviços comunicam entre si através de uma **rede interna** definida no ficheiro `docker-compose.yml`, assegurando o correto funcionamento coordenado da arquitetura. Esta abordagem torna os serviços — especialmente a **interface gráfica** (frontend) — facilmente acessíveis ao utilizador final, de forma local ou remota, conforme configurado.

5 - Testes e resultados

Abaixo, seguem-se os testes realizados sobre os algoritmos utilizados no projeto final, bem como o ambiente utilizado para testá-los e as suas condições.

5.1 - Configurações do testes

Os testes realizados para avaliar o desempenho de cada algoritmo foram realizados usando funcionalidades da própria aplicação, considerando que este é o objetivo final da mesma. Para tal, foram realizados testes com a configuração original de empregados-equipas-preferências exposta no problema original do caso de estudo fornecido pela empresa solicitante do projeto, sendo essa uma configuração de 12 empregados, distribuídos irregularmente entre as equipas A e B (tendo a equipa A mais empregados que a equipa B) e outra configuração com 30 empregados distribuídos de forma igualitária entre as duas equipas. Foram também usados templates de férias de 12 empregados e outro de 30 empregados para os testes e templates de mínimos apropriados para cada caso com valores de mínimos diários ajustados a cada caso. Para o tempo máximo, foram escolhidos 10 minutos para testes com 12 empregados e 20 minutos para testes com 30 empregados, considerando que o aumento do número de empregados pode aumentar o tempo de procura por uma solução, apesar de que cada algoritmo pode encontrar uma solução antes do tempo previsto.

5.2 - Avaliação dos resultados

Para avaliar os resultados de cada horário gerado, foram analisados diretamente os resultados das análises dos Key Performance Indicators (KPIs), visíveis tanto na página de visualização de um horário específico quanto na página de comparação de dois horários. Entre todos os KPIs analisados, foram individualmente selecionados para serem os principais para a visualização em gráficos os seguintes:

1. Mínimos que não foram cumpridos
2. Shift balance
3. Two Teams Preference Level

Já que a maioria dos algoritmos conseguem cumprir totalmente os demais KPIs relacionados às restrições do problema original para quase toda configuração de empregados-equipas, tempo máximo ou templates possível, estes 3 são métricas com valores que tendem a variar bastante dependendo das configurações utilizadas. Daí a sua escolha.

5.3 - Testes

5.3.1 - Testes de performance dos mínimos

Neste teste foi avaliado se os horários gerados conseguiram cumprir os mínimos diários exigidos para cada equipa e turno sendo cada falha contabilizada como uma métrica negativa já que representa falta de pessoal numa determinada posição do dia. O

objetivo foi observar o desempenho dos algoritmos nesse aspeto e perceber em que situações essas falhas ocorrem mais frequentemente. Os resultados obtidos estão refletidos nos gráficos apresentados onde é possível visualizar a distribuição e quantidade de falhas para cada caso de configuração de empregados.

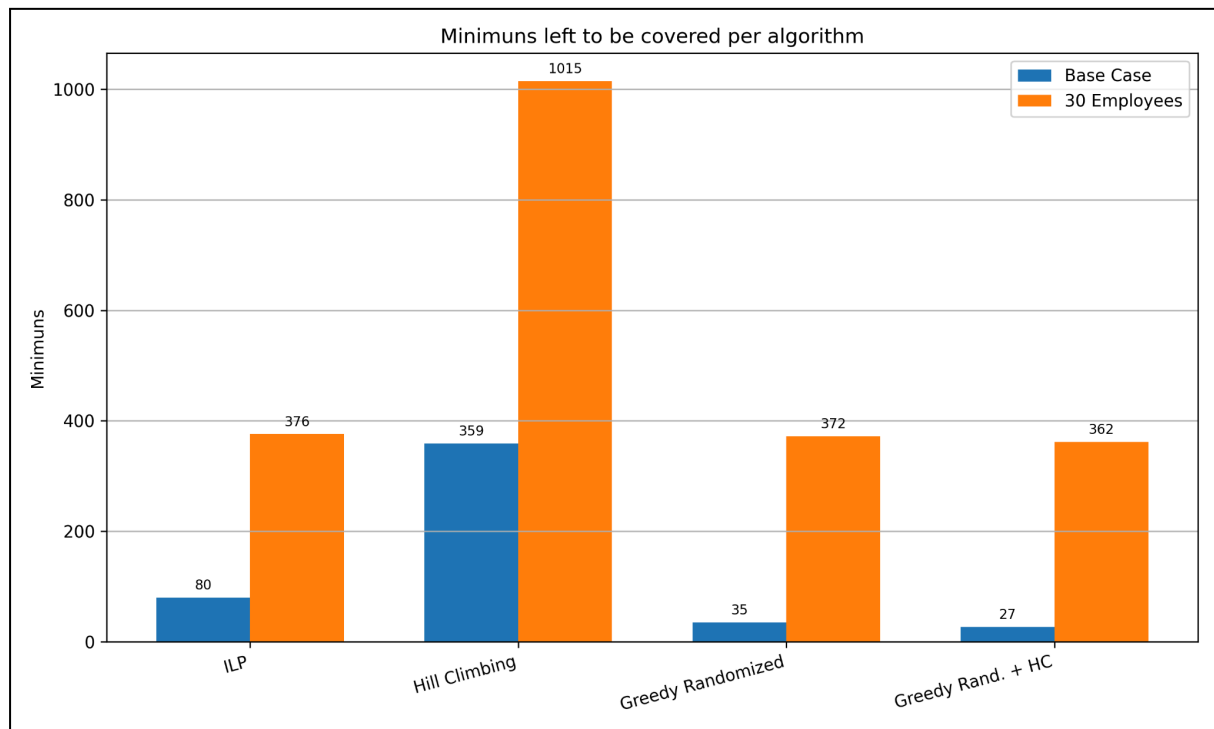


Figura 29: Mínimos que faltaram a serem cumpridos

5.3.2 - Testes de performance dos shift balance e two team preference level

Este teste focou-se em verificar se os turnos foram distribuídos de forma equilibrada entre os trabalhadores e se as preferências dos empregados que pertencem a mais de uma equipa foram respeitadas, garantindo assim uma melhor justiça e bem-estar dos colaboradores além de contribuir para uma melhor organização das equipas e maior eficiência na execução das tarefas. Os efeitos destas métricas podem ser observados nos diagramas incluídos que ilustram o grau de equilíbrio entre turnos e o nível de cumprimento das preferências de equipa.

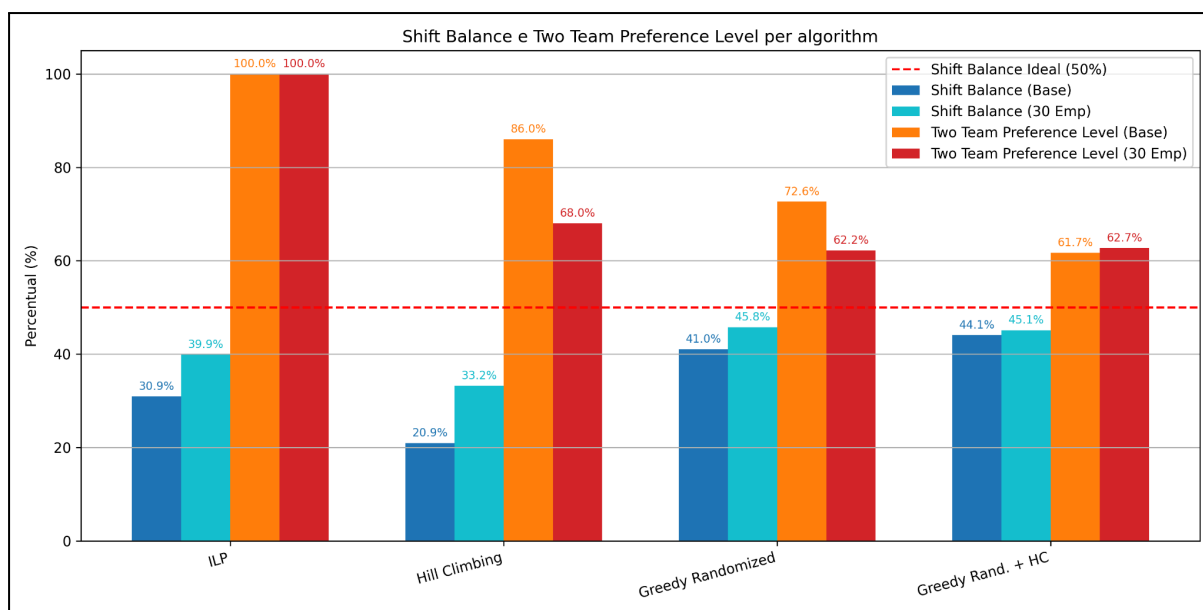


Figura 30: Shift Balance e Two Teams Preference Level

5.3.3 - Testes de cumprimento de KPIs em função do tempo máximo

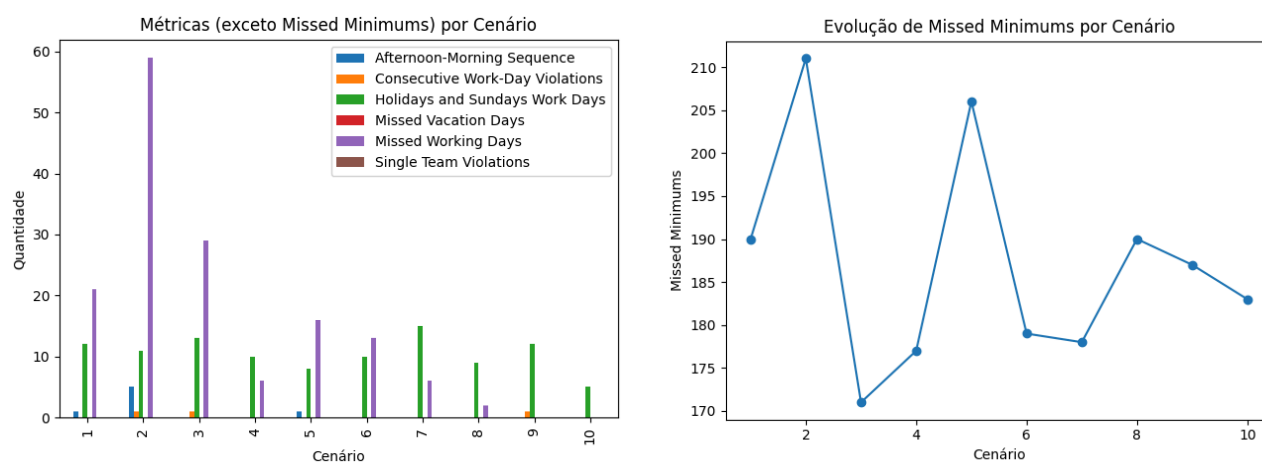


Figura 31: Evolução dos KPIs ao longo do tempo

O gráfico mostra a evolução do Hill Climbing ao longo do tempo, e podemos observar que o gráfico ilustra a progressão do algoritmo Hill Climbing durante 10 minutos de execução num cenário com 12 funcionários. Observa-se uma melhoria constante no valor do indicador ao longo do tempo, demonstrando que o algoritmo é eficaz em otimizar a solução mesmo a partir de diferentes pontos de partida. A solução inicial é sempre gerada de forma aleatória, o que significa que o valor inicial do Hill Climbing pode variar entre execuções. No entanto, todas as execuções mantêm constantes os mesmos funcionários, preferências de equipa e templates de férias, assegurando que as

restrições do problema não são alteradas. Apesar da variabilidade inicial, o algoritmo consegue refinar continuamente a solução, convergindo para resultados de maior qualidade. Isso evidencia a robustez e consistência do método, que é capaz de encontrar boas soluções de forma progressiva independentemente da configuração inicial.

6 - Conclusão

6.1 - Conclusão e resumo do projeto

O projeto SmarTask demonstrou-se uma solução eficaz para o problema complexo da geração automática de horários de trabalho, enfrentado por organizações com elevada variabilidade de turnos e preferências de equipa. Ao longo do desenvolvimento, foi possível desenhar, implementar e avaliar uma plataforma capaz de integrar múltiplos algoritmos de geração de horários, os quais respeitam um conjunto abrangente de restrições legais e operacionais. O sistema criado distingue-se das soluções comerciais existentes pela sua flexibilidade, extensibilidade e capacidade de comparação entre algoritmos. Foram desenvolvidos e testados vários métodos, desde abordagens determinísticas como a Programação Linear Inteira (ILP), até algoritmos estocásticos como Hill Climbing e Greedy Randomized com refinamento, permitindo aferir o desempenho de cada um em diferentes contextos. Além da robustez da infraestrutura técnica composta por um frontend reativo, API modular e integração assíncrona via message broker a plataforma revelou-se escalável e adaptável, suportando diferentes cenários de entrada e validando a geração de horários através de KPIs predefinidos. Este trabalho valida a viabilidade de utilização de técnicas de otimização e inteligência artificial na geração automática de escalas, contribuindo para a redução de esforço manual, melhoria da justiça na alocação de turnos e conformidade legal. O projeto atingiu com sucesso os objetivos definidos inicialmente, fornecendo uma base sólida para evoluções futuras.

6.2 - Trabalho futuro

Apesar da maioria dos objetivos iniciais e mais fundamentais terem sido cumpridos, alguns aspetos que foram planeados e outros que vieram a surgir ainda não foram implementados completamente ou parcialmente. Abaixo, segue-se destacada a lista de features que foram planeadas e que não foram implementadas no final do projeto, em função do tempo limitado, e outra lista com algumas possíveis features futuras :

1. Features não implementadas:
 - a. Adicionar novos algoritmos dinamicamente na aplicação como ficheiros .py para serem utilizados como métodos de geração de horário;
 - b. Importar esses novos métodos de forma prática, de modo a poder integrá-los no sistema;
 - c. Remover métodos existentes;
2. Features para o futuro do projeto
 - a. Geração de gráficos que possam compilar visualmente a evolução de KPIs para um método ou para 2 métodos (usando horários existentes);

- b. Suporte para visualização de exceções que causem falhas na geração de horários no histórico de tarefas;
- c. Página de logs que mostram todas as tarefas e seus respectivos estados(contendo apenas completed ou failed);

6.3 - Repositórios e Instruções

O código fonte do projeto está presente na organização do GitHub criada especialmente para este projeto, o qual contém todos os repositórios utilizados no projeto:

Link da organização: <https://github.com/PI-SmartTask>

Repositório específico do código fonte:

Link da Projeto: <https://github.com/PI-SmartTask/SmartTask>

Instruções de instalação, também presentes no **README.md** do repositório do código fonte:

Nota Técnica:

Este projeto foi desenvolvido e testado em sistemas operacionais da família Unix, especificamente Ubuntu e Arch Linux. Não foram realizados testes de funcionamento em outros sistemas operacionais, como Windows ou MacOS. Caso haja problemas de compatibilidade em outros ambientes, poderá ser necessária uma investigação adicional para adaptar o sistema ao novo ambiente.

Requisitos:

É necessário possuir o **Docker Compose** instalado localmente. Os comandos abaixo podem variar conforme o sistema operacional.

Passos para execução do ambiente de desenvolvimento:

1. Acesse o diretório raiz do projeto.
2. Execute o comando para construir as imagens e iniciar os serviços em modo de desenvolvimento:
 - a. `docker-compose up --build`
3. Após a execução, a aplicação frontend poderá ser acessada pelo navegador através do endereço:
 - a. <http://localhost:5173/>
 - b. Login com:

- i. Email : manager
 - ii. Password : manager
- 4. Para encerrar a aplicação, utilize o atalho de interrupção no terminal (geralmente **Ctrl+C**) e, em seguida, pare e remova os containers com o comando:
 - a. **docker-compose down**
- 5. Caso seja necessário realizar o debug de um container ou serviço específico, utilize o seguinte comando:
 - a. **docker-compose logs [nome-do-serviço]**